UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

CARRERA DE INGENIERÍA EN SISTEMAS / INGENIERÍA INFORMÁTICA

SISTEMAS DE INFORMACIÓN II (INF-412) — SEMESTRE 2-2026

MIRROR-IA

Plataforma SaaS Multi-tenant de Comercio Omnicanal y Probador Virtual Inteligente para Boutiques de Moda Femenina

Documento Académico Exhaustivo de Especificación de Requisitos, Análisis y Diseño Arquitectónico Orientado a Objetos (ICONIX Process & UML 2.5), Resiliencia Transaccional Offline-First (Web PWA + Móvil Expo) y Machine Learning Supervisado / No Supervisado con Fallback Heurístico.

Docente Guía:	MSc. Angélica Garzón Cuellar
Asignatura:	Sistemas de Información II (INF-412) — Grupo SA
Evaluación:	Proyecto de Examen Parcial 2 (Proceso ICONIX, UML 2.5 e Implementación)
Boutique Piloto:	Tiendaria (Slug: `tiendaria`) — Catálogo y Datos Reales
Gestión:	Semestre 2-2026
Ubicación:	Santa Cruz de la Sierra, Bolivia — Noviembre 2026

ÍNDICE GENERAL DE CONTENIDOS

Capítulo I: Aspectos Generales

Capítulo II: Análisis de Requisitos y Modelado de Casos de Uso

Capítulo III: Análisis y Diseño Orientado a Objetos (ICONIX)

Capítulo IV: Desarrollo e Implementación Técnica

Capítulo V: Pruebas y Resultados de Validación

Capítulo VI: Conclusiones y Recomendaciones

Capítulo VII: Bibliografía y Anexos

CAPÍTULO I. ASPECTOS GENERALES

1.1. Introducción

En la actualidad, la industria del comercio minorista y, de manera particular, el sector de la moda femenina (*fashion retail*), experimenta una acelerada transformación impulsada por la digitalización de los canales de venta, la exigencia de experiencias omnicanal fluidas y la necesidad de optimizar la gestión operativa en tiempo real. La convergencia entre el comercio electrónico tradicional y los puntos de venta físicos (POS) demanda plataformas capaces de garantizar disponibilidad continua, fidelidad estética de marca y personalización avanzada.

Durante la primera etapa de desarrollo (Primer Parcial de Sistemas de Información II), se diseñó e implementó la línea base funcional de la plataforma de comercio electrónico denominada **MirrorIA**, orientada inicialmente a resolver los desafíos comerciales, de inventario y de atención en probador para una tienda física multi-sucursal de moda femenina (hoy preservada como la tienda oficial demo **Tiendaria**, con slug `tiendaria`). Dicha solución inicial integró la administración de catálogos multivariante (categorías, temporadas, colecciones, tallas y colores HEX con variantes SKU), control de existencias con actualización condicional atómica, reservas de prendas para prueba presencial en tienda con reglas anti-acaparamiento, registro de compras presenciales de mostrador y un probador virtual interactivo basado en inteligencia artificial generativa con Decart (`lucy-vton-3.5`).

No obstante, las dinámicas del mercado del software contemporáneo y las exigencias de escalabilidad empresarial demandan una evolución estructural del sistema: la transición desde una aplicación monotienda hacia una **plataforma SaaS (*Software as a Service*) Multi-tenant**. Bajo este nuevo paradigma, múltiples boutiques de moda independientes pueden registrarse, aprovisionar su propio espacio de trabajo (*workspace*) en cuestión de segundos, personalizar integralmente su identidad visual corporativa conforme a estándares de accesibilidad internacional, operar de manera aislada y segura sobre una base de datos relacional compartida, y gestionar sus cobranzas tanto digitales (Stripe y QR interoperable) como físicas.

Asimismo, ante la realidad de la infraestructura de telecomunicaciones en Bolivia y la región, donde la conectividad a Internet en tiendas físicas y centros comerciales presenta intermitencias o cortes imprevistos, se vuelve imperativo dotar a la plataforma de capacidades **Offline-First**. Esto permite que tanto las clientas como el personal de caja continúen navegando catálogos y procesando transacciones sin conexión desde aplicaciones web progresivas (PWA) y dispositivos móviles nativos, garantizando sincronización idempotente y libre de sobreventas al restablecer la red. Finalmente, la incorporación de consolas diferenciadas de Inteligencia de Negocios (BI) y modelos de Machine Learning desacoplados atados a Indicadores Clave de Desempeño (KPIs) completa la transformación de MirrorIA en una solución de vanguardia tecnológica para el sector textil.

1.2. Planteamiento del Problema

El sector minorista de indumentaria femenina en Bolivia enfrenta diversas barreras tecnológicas y operativas que limitan su competitividad frente a grandes cadenas internacionales:

1. **Altos costos y complejidad de digitalización:** Pequeñas y medianas boutiques carecen de capital para desarrollar plataformas de comercio electrónico propias con probadores virtuales y analítica avanzada. Al recurrir a soluciones genéricas, pierden su identidad visual de marca o sufren limitaciones para gestionar sucursales físicas distribuidas.
2. **Vulnerabilidad operativa por dependencia de conectividad:** La mayoría de los sistemas de punto de venta (POS) y tiendas web dependen de una conexión a Internet permanente e ininterrumpida. Cuando la red cae en el mostrador físico, se detiene la facturación, se pierde la visibilidad de existencias y se generan colisiones de inventario o doble facturación al intentar cobrar manualmente.
3. **Pérdida de ventas por incertidumbre en tallas y acaparamiento:** El comercio electrónico de moda sufre tasas de devolución de hasta el 30% debido a que las compradoras no pueden visualizar cómo luce la prenda en su propio cuerpo. A su vez, los sistemas tradicionales de reserva permiten que clientas aparten prendas indefinidamente sin presentarse a la tienda, inmovilizando inventario valioso (*no-show*).
4. **Falta de herramientas analíticas accesibles para gerentes no técnicos:** Los dueños de boutiques suelen tomar decisiones de abastecimiento y pedidos a talleres de confección por intuición, careciendo de pronósticos matemáticos de demanda a 30 días, segmentación conductual de clientas o alertas tempranas de descuadres transaccionales en caja.

Ante esta problemática, surge la interrogante central de ingeniería: **¿Cómo estructurar y desarrollar una plataforma SaaS multi-tenant omnicanal para el sector de la moda femenina que garantice aislamiento de datos, resiliencia transaccional offline-first, personalización estética accesible, probador virtual generativo y analítica predictiva mediante Machine Learning?**

1.3. Objetivos del Proyecto

1.3.1. Objetivo General

Desarrollar e implementar **MirrorIA**, una plataforma de comercio electrónico bajo el modelo **SaaS Multi-tenant** para el sector de moda femenina, aplicando la metodología **ICONIX** y estándares **UML 2.5.1**, que integre capacidades de operación **Offline-First** (PWA y móvil con persistencia local e idempotencia), pasarelas de pago enchufables (Stripe y Wayra QR), probador virtual con inteligencia artificial generativa, consolas de Business Intelligence (BI) por rol y modelos de Machine Learning orientados a optimizar la toma de decisiones comerciales y logísticas.

1.3.2. Objetivos Específicos

1. **Modelar e implementar la arquitectura SaaS Multi-tenant:** Diseñar una base de datos relacional compartida en PostgreSQL con discriminador `tenant_id` en las tablas de negocio, reforzada mediante políticas de contexto por request (`X-Tenant`), que garantice el aprovisionamiento automatizado de tiendas y el aislamiento lógico absoluto entre inquilinos.

2. **Desarrollar el subsistema de personalización de identidad de marca:** Construir un motor de theming dinámico basado en tokens de diseño que permita a cada boutique parametrizar logotipos, tipografías, modos visuales (`CLARO` / `OSCURO`) y paletas cromáticas validadas matemáticamente bajo el estándar de accesibilidad **WCAG 2.1** (ratio de contraste $\ge 4.5:1$).
3. **Construir el núcleo de catálogo multivariante y gestión de inventario condicionado:** Implementar la administración jerárquica de categorías, temporadas, colecciones, prendas maestras y variantes SKU por talla/color con precio en centavos, asegurando rebajas atómicas de stock libre mediante sentencias `UPDATE` condicionadas para evitar cantidades negativas.
4. **Implementar el flujo omnicanal de reservas físicas con reglas anti-acaparamiento:** Diseñar una máquina de estados finitos (`PENDIENTE` $\to$ `CONFIRMADA` $\to$ `EN_TIENDA` $\to$ `COMPLETADA` / `NO_SHOW`) que bloquee stock en `cantidad_reservada` (hasta máx. 50% de existencias libres) y aplique penalizaciones automáticas de 30 días a clientas con inasistencias reiteradas.
5. **Integrar el probador virtual con inteligencia artificial generativa:** Conectar el sistema con el motor Decart (`lucy-vton-3.5`) para la síntesis de video/imagen en tiempo real de prendas sobre la imagen de la usuaria, mediante la generación de credenciales efímeras seguras de 5 minutos y control de tasa por IP.
6. **Diseñar una arquitectura de pasarelas de pago enchufables:** Implementar el registro centralizado `RegistroPasarelasService` bajo la interfaz `IPasarelaProveedor` , soportando pagos internacionales con **PayPal** (Orders v2: la orden se captura en el servidor al volver la clienta) y con tarjeta (Stripe Checkout con webhooks firmados), pagos nacionales con QR (Wayra, un banco **simulado** con avisos autenticados por HMAC-SHA256, elegido porque la API de un banco boliviano real no es accesible para este proyecto académico) y cobro manual en efectivo en mostrador.
7. **Desarrollar la resiliencia transaccional Offline-First e idempotencia:** Configurar un Service Worker con cache App Shell e IndexedDB (TTL 7 días) en la PWA web, y diseñar el almacenamiento local embebido en SQLite para la aplicación móvil nativa (Expo), incorporando colas de salida (*outbox*) FIFO y la cabecera `Idempotency-Key` (UUIDv4 con hash SHA-256 canónico) con resolución determinista del conflicto `stock = 1` mediante transacciones compensatorias.
8. **Diseñar consolas diferenciadas de Inteligencia de Negocios (BI):** Construir el Dashboard Ejecutivo para el rol `ADMIN` (ingresos con deltas simétricos, rotación de percha, tasa de conversión probador a venta, cuota de pasarelas y consumo de cuotas del plan SaaS) y el Dashboard Operativo POS para `CAJERO` / `ENCARGADO` (corte diario local en `America/La_Paz` , alertas de stock crítico ≤ 2 u. y agenda cronológica de citas con atención en 1 clic).
9. **Implementar el motor de reportes dinámicos con IA y exportación contable a Excel:** Desarrollar un agente conversacional basado en DeepSeek LLM que mapee lenguaje natural (texto o voz) a un catálogo formal de 23 métricas parametrizadas (sin generar SQL directo), permita simulaciones *What-If* en memoria con DuckDB y genere libros corporativos `.xlsx` con formato contable `"Bs" #,##0.00` y fórmulas de totales mediante `exceljs` .
10. **Desplegar un microservicio autónomo de Machine Learning orientado a KPIs:** Construir un servicio desacoplado en Python FastAPI (puerto `8002` , contenedor `mirroria_ml`) con 4 modelos estadísticos reproducibles ($N=10.000$, `seed=42`): Random Forest para predicción de demanda a 30 días, Regresión Logística L2 para riesgo de No-Show, K-Means para segmentación RFM e Isolation Forest para auditoría de anomalías en caja POS, enlazado al backend NestJS mediante un proxy con fallback heurístico resiliente (`X-ML-Fallback: true`).

1.4. Justificación

1.4.1. Justificación Técnica

La plataforma implementa un diseño de software moderno sustentado en la metodología **ICONIX**, garantizando consistencia y trazabilidad rigurosa desde la captura de requisitos hasta el código ejecutable. La adopción de un **Monolito Modular** en NestJS 12 (TypeScript con ESM nativo) permite un estricto desacoplamiento entre módulos mediante servicios exportados e identificadores planos `UUID`, facilitando la futura migración hacia microservicios. A su vez, el aislamiento multi-tenant en una base de datos compartida en PostgreSQL 16 optimiza el uso de memoria RAM (crítico en entornos de desarrollo con recursos limitados) frente al enfoque de base de datos por inquilino. La combinación de PWA con IndexedDB, SQLite móvil, tokens criptográficos JWT y validación matemática de contraste accesible WCAG 2.1 posiciona al proyecto en los más altos estándares de calidad técnica e ingeniería de software.

1.4.2. Justificación Económica y de Negocio

Bajo el modelo SaaS, MirrorIA democratiza el acceso a tecnología de primer nivel para micro, pequeñas y medianas empresas textiles. En lugar de desembolsar miles de dólares en desarrollo a medida, las boutiques acceden a planes mensuales escalables (**Básico** Bs 99, **Crecimiento** Bs 249 y **Cadena** Bs 599) que crecen con su volumen de ventas. La integración de pasarelas electrónicas (Stripe y Wayra QR) reduce el costo y tiempo de cobranza, mientras que la analítica de Machine Learning y la prevención de *no-show* maximizan la rentabilidad del inventario y reducen mermas por prendas inmovilizadas.

1.4.3. Justificación Social y Operativa

Para las consumidoras finales, la plataforma proporciona una experiencia de compra personalizada, segura y empática, permitiendo comprobar el calce de las prendas mediante inteligencia artificial generativa y apartar citas de prueba física en sucursal sin pagos previos forzosos. Para el personal de tienda y caja, la consola operativa y la resiliencia offline eliminan la frustración provocada por caídas de red, agilizando la atención al cliente y asegurando que ninguna venta se pierda por problemas de conectividad.

1.5. Alcance y Limitaciones

1.5.1. Alcance del Proyecto

El proyecto abarca el ciclo de vida completo de análisis, diseño, desarrollo, pruebas y despliegue del 2.º Ciclo de Sistemas de Información II:

- **Gobierno SaaS y Aprovisionamiento:** Registro público de boutiques, asignación de planes, control de cuotas máximas de recursos (`assertDentroDelPlan`), panel de control del Superadministrador con métricas globales (MRR, Churn, GMV) y moderación de tiendas.
- **Personalización de Marca:** Configuración integral de identidad visual por tienda (logo, fuentes, radios de bordes, modo claro/oscuro y paleta de colores con contraste WCAG 2.1 verificado).

- **Comercio Omnicanal y POS:** Catálogo maestro, gestión de existencias por sucursal, órdenes a confeccionistas, carrito persistente `jsonb`, cupones de descuento, checkout digital y ventas presenciales en mostrador con comprobantes institucionales `C-XXXXXXXX`.
- **Probador Virtual y Reservas:** Integración con Decart AI (`lucy-vton-3.5`) y flujo de citas para prueba física en sucursal con máquina de estados y sanción automática por inasistencia.
- **Pasarelas Enchufables:** Procesamiento de pagos con PayPal y Stripe Checkout (internacionales, en modo sandbox/prueba), Wayra QR (nacional, banco simulado con firma HMAC) y efectivo, tanto para ventas de retail como para suscripciones B2B.
- **Operación Offline e Idempotencia:** PWA web (Service Worker + IndexedDB, con pantalla de **Mostrador** para vender en caja sin conexión) y aplicación móvil nativa (Expo + SQLite: catálogo, carrito y compras en cola) con colas outbox FIFO, cabecera `Idempotency-Key` (SHA-256) y resolución de conflictos de concurrencia `stock = 1` mediante transacciones compensatorias.
- **Business Intelligence (BI):** Consola ejecutiva gerencial para `ADMIN` y consola operativa POS para `CAJERO` / `ENCARGADO`.
- **Reportes Dinámicos, Excel y PDF:** Asistente DeepSeek LLM con catálogo de 23 métricas parametrizadas, simulación *what-if* en memoria con DuckDB y exportación a hojas `.xlsx` (`exceljs`) y a **PDF** (`pdf-kit`); la persona puede pedir el formato en la misma pregunta («… en pdf»).
- **Microservicio Machine Learning:** Servicio Python FastAPI (:8002) con 4 modelos entrenados reproducibles (Demanda Random Forest, No-Show Regresión Logística, Segmentación RFM K-Means y Detección de Anomalías Isolation Forest) con proxy NestJS y fallback heurístico.

1.5.2. Limitaciones del Proyecto

- **Entorno Móvil:** La aplicación móvil (React Native / Expo SDK 57) implementa la operación sin conexión con SQLite (catálogo con vigencia de 7 días, carrito local y compras en cola con `Idempotency-Key`), verificada con SQLite real y contra el backend; su prueba en un dispositivo físico queda pendiente. La app es exclusiva para clientas: la venta en caja sin conexión se realiza desde el Mostrador de la PWA.
- **Moneda de Facturación:** Las transacciones locales se calculan y registran exclusivamente en centavos de Bolivianos (BOB), y los pagos con tarjeta internacional vía Stripe se liquidan en su equivalente configurado en centavos.
- **Catálogo Especializado:** La plataforma está orientada exclusivamente a indumentaria y moda femenina; no incluye módulos de gestión para calzado industrial o sastrería masculina a medida.
- **Hardware Fiscal:** El sistema emite comprobantes digitales internos de venta (`C-XXXXXXXX`); la integración con impresoras fiscales térmicas físicas de punto de venta se asume a través de los controladores de impresión nativos del navegador web.

1.6. Metodología de Desarrollo ICONIX

Para guiar el desarrollo de MirrorIA se seleccionó la metodología **ICONIX** (*Use Case Driven Object Modeling with UML*), creada por Doug Rosenberg y Matt Stephens. ICONIX es un proceso ágil, pragmático y guiado por casos de uso que se posiciona entre el enfoque exhaustivo del Proceso Unificado (RUP) y la extrema ligereza del desarrollo ágil no documentado (XP).

El principio medular de ICONIX es mantener una **trazabilidad continua y verificable** que conecta los requisitos expresados en lenguaje natural con el código fuente ejecutable, utilizando un conjunto mínimo y suficiente de diagramas UML 2.5:

Las 4 Fases de ICONIX en MirrorIA

1. **Fase 1 — Análisis de Requisitos:**
2. Se identifican los **Actores** del sistema (humanos y servicios externos) y se elabora el **Modelo de Dominio Preliminar** como un glosario visual de conceptos de negocio (Inquilino, Plan, Prenda, Sucursal, Venta, Reserva, Pago).
3. Se redactan los **Casos de Uso Esenciales** (17 CUs consolidados) en lenguaje de usuario, evitando terminología de interfaz técnica.
4. Culmina con la **Revisión de Requisitos (RR)** para asegurar que el alcance cubra las exigencias académicas de SI2.
5. **Fase 2 — Análisis y Diseño Preliminar (El puente de robustez):**
6. Se elaboran los **Diagramas de Robustez** desglosando cada caso de uso en objetos estereotipados:
   - **Boundary (Frontera):** Interfaz visible (`PantallaWeb...` y `PantallaMovil...`).
   - **Control (Controlador):** Lógica de orquestación, validación y aplicación de reglas de negocio (`ServicioVentas`, `ValidadorTenancy`).
   - **Entity (Entidad):** Datos de negocio persistentes (`Venta`, `InventarioSucursal`).
7. Se actualiza el modelo de dominio agregando atributos y asociaciones descubiertas durante el análisis de robustez.
8. Concluye con la **Revisión Preliminar de Diseño (PDR)**.
9. **Fase 3 — Diseño Detallado:**
10. Se transforman los diagramas de robustez en **Diagramas de Secuencia UML 2.5**, asignando formalmente los métodos HTTP, llamadas asíncronas, cabeceras `Idempotency-Key` y respuestas de error.
11. Se consolida el **Diagrama de Clases de Diseño**, detallando tipos de datos TypeScript/Python, visibilidad y métodos.
12. Se diseña el esquema relacional de base de datos en PostgreSQL con restricciones de integridad y multitenancy.
13. Finaliza con la **Revisión Crítica de Diseño (CDR)**.
14. **Fase 4 — Implementación y Pruebas:**
15. Codificación del sistema sobre la arquitectura monorepo: backend en NestJS 12, microservicio de ML en FastAPI y frontend en React 19 PWA.
16. Ejecución de baterías de pruebas automatizadas: 557 tests unitarios en backend, 345 tests en frontend, 17 en la app móvil, pruebas de integración e2e y scripts de QA de aislamiento multi-tenant y reglas de mostrador.

CAPÍTULO II. ANÁLISIS DE REQUISITOS

2.1. Descripción del Sistema Propuesto

MirrorIA es una plataforma de comercio electrónico bajo la modalidad **SaaS (*Software as a Service*) Multi-tenant** especializada en el sector de la moda femenina (*fashion retail*), diseñada para que múltiples boutiques independientes administren sus catálogos, sucursales físicas, inventarios, reservas de probador y ventas de forma estrictamente aislada y segura.

Su arquitectura utiliza un backend desarrollado en **NestJS 12** (Node.js 24, TypeScript ESM) y una base de datos relacional compartida en **PostgreSQL 16**, donde el aislamiento lógico entre boutiques se garantiza mediante la inyección obligatoria del discriminador `tenant_id` en las 22 tablas del dominio transaccional, verificado por interceptores globales (`TenantInterceptor`) y contexto asíncrono (`AsyncLocalStorage`). El acceso a los recursos y la capacidad operativa están regulados por planes comerciales de suscripción (**Básico**, **Crecimiento** y **Cadena**), los cuales gobiernan los límites máximos de sucursales físicas, productos activos en vitrina y personal operativo de staff, así como el nivel de personalización de marca (`ESENCIAL`, `COMPLETA` y `MARCA_BLANCA`).

Para operar de manera continua ante caídas o intermitencias en la conexión a Internet en tiendas físicas y centros comerciales, MirrorIA implementa una arquitectura **Offline-First**: 1. **Cliente Web:** Funciona como una **Aplicación Web Progresiva (PWA)** desarrollada en **React 19**, utilizando Service Workers para el almacenamiento en caché del App Shell y la base de datos local **IndexedDB** (`mirroria_offline_db`) con un tiempo de vida (TTL) de 7 días para el catálogo y existencias locales. El punto de venta (POS) y la tienda web permiten registrar ventas presenciales de contado y pedidos sin conexión. 2. **Cliente Móvil:** Diseñado sobre **React Native / Expo SDK 57**, utiliza una base de datos local embebida **SQLite** (`expo-sqlite`) como réplica espejo del catálogo e inventario.

Todas las operaciones registradas sin conexión se identifican mediante identificadores universales únicos (**UUIDv4**), se encolan en almacenes locales de salida (*outbox*) bajo una política FIFO y se sincronizan de forma asíncrona hacia el backend central mediante la cabecera canónica `Idempotency-Key` respaldada por funciones criptográficas de dispersión (SHA-256). Ante colisiones de concurrencia sobre la última prenda disponible (`stock = 1`), el servidor actúa como autoridad única emitiendo un error tipado HTTP 409 (`STOCK_AGOTADO_CONFLICTO`), ante el cual los clientes ejecutan una **transacción compensatoria** de reversión local y despliegan interfaces interactivas con alternativas inmediatas guiadas.

Asimismo, la plataforma integra una experiencia omnicanal diferenciada: un **probador virtual generativo** basado en inteligencia artificial (Decart `lucy-vton-3.5`) consumido mediante tokens efímeros seguros; consolas de **Business Intelligence (BI)** adaptadas al rol del usuario (**Consola Ejecutiva** para dueños y **Consola Operativa POS** con agenda de citas y alertas de stock crítico para cajeros); un motor de **reportes dinámicos** que interpreta lenguaje natural (texto o voz) mediante DeepSeek LLM sin generar SQL directo, con simulaciones *What-If* en memoria (DuckDB) y exportación nativa a hojas contables `.xlsx` (`exceljs`); y un microservicio independiente de **Machine Learning** en Python FastAPI (puerto `8002`, contenedor `mirroria_ml`) con 4 modelos es-

tadísticos reproducibles ($N=10.000$, `seed=42`) para proyección de demanda, estimación de inasistencia (*no-show*), segmentación RFM y detección preventiva de anomalías transaccionales en caja.

2.2. Identificación de Actores y Casos de Uso

En el marco de la metodología ICONIX y la arquitectura SaaS multi-tenant omnicanal de MirrorIA, se identifican seis actores principales con responsabilidades y niveles de acceso criptográfico diferenciados:

2.2.1. Actores

- **SuperAdmin (Dueño de la Plataforma SaaS — `DUENO_PLATAFORMA`):** Actor con privilegios globales sobre toda la infraestructura de MirrorIA (`tenant_id = NULL`). Es el responsable de registrar y administrar los planes comerciales de suscripción (Básico, Crecimiento y Cadena), supervisar indicadores clave de desempeño corporativo (MRR, ARR, Churn Rate, GMV y bitácora de eventos), auditar inquilinos, conceder extensiones de períodos de prueba (`TRIAL`), suspender tiendas morosas y restablecer identidades visuales infractoras de oficio. No interviene en la operativa comercial ni en las existencias de ninguna boutique.
- **Admin (Dueño de Boutique / Tenant Admin — `ADMIN`):** Usuario con máxima jerarquía dentro de su respectiva empresa contratante. Administra la red de sucursales físicas (respetando las cuotas de su plan), da de alta al personal operativo asignándole tienda fija (`CAJERO`, `ENCARGADO_SUCURSAL`), administra el catálogo maestro de prendas multivariante, personaliza la identidad visual corporativa (logo, fuentes, banners, modo claro/oscuro y colores con contraste accesible **WCAG 2.1**), contrata y renueva la suscripción B2B mediante Stripe o Wayra QR, monitorea la Consola Ejecutiva de BI y toma decisiones asistido por Machine Learning y reportes dinámicos.
- **Encargado de Sucursal (`ENCARGADO_SUCURSAL`):** Usuario responsable del control logístico y comercial en una tienda física asignada (`sucursal_id` forzado en JWT). Administra el inventario local, supervisa el kardex inmutable de movimientos, emite y recepciona órdenes de compra a talleres de confección, efectúa ajustes justificados de existencias (mermas o roturas), gestiona el avance de las citas de probador (`CONFIRMADA`, `EN_TIENDA`) y supervisa la conciliación de conflictos de stock derivados de sincronizaciones offline.
- **Cajero (`CAJERO`):** Personal operativo asignado al punto de venta (POS) y mostrador físico de la sucursal. Registra ventas directas presenciales con descuento atómico de existencias y emisión de comprobantes `C-XXXXXXXX`, confirma cobros en efectivo de pedidos web y reservas, y opera la Consola Operativa POS del día con corte en huso local `America/La_Paz`, atendiendo citas de clientas en un solo clic y visualizando alertas de prendas en stock crítico (≤ 2 unidades).
- **Proveedor:** Entidad externa vinculada a la cadena de suministro textil de una boutique específica. Provee lotes de prendas confeccionadas, materias primas o insumos registrados en el sistema, permitiendo el abastecimiento de mercancía mediante órdenes de compra con control de stock en tránsito. En el alta de cada boutique, el sistema crea automáticamente el proveedor "Producción propia".
- **Cliente (`CUSTOMER`):** Usuaria final consumidora de la boutique, registrada exclusivamente en el espacio de trabajo de la tienda navegada (`/t/:slug`). Interactúa vía web (PWA) o aplicación móvil nativa (Expo) para explorar catálogos adaptados a la marca, probarse prendas mediante el Probador Virtual por IA (Decart `lucy-vton-3.5`), apartar reservas de prendas para prueba presencial sin pago previo, gestionar su carrito de compras, aplicar cupones de descuento y pagar electrónicamente (tarjeta Stripe o QR interoperable

Wayra). En caso de corte de red, puede continuar navegando catálogos cacheados y registrando reservas o compras de forma diferida.

2.2.2. Casos de Uso del 2.º Ciclo por Módulo Funcional

Los casos de uso del 2.º Ciclo consolidan los servicios de valor del sistema, agrupados en los cinco módulos funcionales rectores de la plataforma:

Módulo 1: Gestión Multitenant y Suscripciones (SaaS)

- **CU01: Aprovisionar Tienda y Workspace SaaS:** Permite al Visitante o futuro Dueño registrar su boutique en modalidad de autoservicio seleccionando un plan comercial. El sistema crea en una sola transacción atómica el registro del inquilino (`inquilinos`), su período de prueba (`TRIAL` de 14 días), el usuario Administrador inicial, la sucursal física matriz, el proveedor propio y el catálogo demo, garantizando el aislamiento lógico con `tenant_id`.
- **CU02: Administrar Suscripciones, Planes y Moderación SaaS:** Permite al SuperAdmin parametrizar precios y cuotas máximas de recursos de los planes (Básico, Crecimiento y Cadena), auditar el MRR y suspender tiendas morosas; y permite al Admin de la boutique consultar el consumo de recursos de su plan (`UsoPlanService`) y pagar o renovar su suscripción mediante pasarelas B2B (Stripe y Wayra QR).
- **CU03: Personalizar Identidad Visual de Boutique:** Permite al Admin configurar la marca corporativa de su tienda (logotipo, tipografías, favicon, modo claro/oscuro y paleta cromática), validando forzosamente que el contraste entre el color principal y el fondo de la tienda cumpla el estándar internacional **WCAG 2.1** (ratio de contraste $\ge 4.5:1$).

Módulo 2: Catálogo, Inventario y Puntos de Venta (POS)

- **CU04: Gestionar Catálogo Maestro y Variantes SKU:** Permite al Admin mantener la taxonomía de moda femenina (categorías con slug amigable, temporadas comerciales, colecciones vinculadas a proveedores, tallas ordenadas y colores normalizados con código HEX), publicando prendas maestras con imágenes `jsonb` y combinaciones SKU con precio en centavos (`precioCents`), respetando el límite de prendas activas de su plan.
- **CU05: Gestionar Inventario, Sucursales y Abastecimiento:** Permite al Admin y al Encargado de Sucursal administrar la red de tiendas físicas, emitir órdenes de compra a confeccionistas incrementando el stock en tránsito, recepcionar mercancía en tienda aumentando atómicamente la `cantidad_disponible` y registrar ajustes manuales justificados con asiento inmutable en kardex.
- **CU08: Procesar Cobros con Pasarelas Enchufables:** Permite procesar los pagos derivados de ventas presenciales o web a través del registro central `RegistroPasarelasService` (`IPasarelaProveedor`), canalizando pagos internacionales vía PayPal (orden aprobada por la clienta y capturada por el servidor) y con tarjeta vía Stripe (sesión alojada, webhook firmado y confirmación al regreso), pagos nacionales con QR dinámico vía el banco simulado Wayra (aviso autenticado por HMAC-SHA256 e idempotencia por `event_id`) y cobro manual en efectivo en mostrador.

Módulo 3: Experiencia del Cliente y Probador Virtual por IA

- **CU06: Explorar Vitrina y Probar Prendas con IA (VTON):** Permite a la Clienta navegar el catálogo público inyectando en tiempo real la identidad visual de la boutique elegida, consultar existencias físicas desglosa-

das por sucursal y proyectar prendas seleccionadas sobre su propia imagen o modelo en tiempo real mediante el Probador Virtual por IA generativa (Decart `lucy-vton-3.5`) a través de tokens efímeros de 5 minutos (10 peticiones/10 min por IP).

- **CU07: Gestionar Carrito y Procesar Checkout Digital:** Permite a la Clienta mantener una bolsa de compras persistente (`jsonb`, máximo 3 unidades por variante SKU), validar cupones de descuento protegidos contra ataques de fuerza bruta y transformar el carrito en una venta en estado `PENDIENTE` que descuenta existencias de forma atómica (máximo 2 compras pendientes sin pagar por usuaria).
- **CU09: Gestionar Reservas para Probador Físico:** Permite a la Clienta apartar hasta 3 prendas para prueba física en sucursal en una ventana entre +30 min y +48 h sin pago previo, bloqueando existencias en `cantidad_reservada` (hasta máx. 50% de stock libre); y permite al personal avanzar la cita (`CONFIRMADA`, `EN_TIENDA`), recepcionarla en un clic en el POS y aplicar sanción automática de 30 días ante 2 inasistencias acumuladas (`NO_SHOW`).

Módulo 4: Operación Offline-First y Sincronización

- **CU10: Operar en Modo Desconectado con Sincronización Segura:** Permite a la Clienta consultar el catálogo y el detalle de las prendas sin conexión (caché por tienda del Service Worker en la PWA, SQLite en la app móvil) y comprar desde el móvil dejando la compra en cola, y al Cajero registrar ventas de contado en el Mostrador sin conexión a Internet (las reservas sí requieren conexión), encolándolas en la cola de salida (*outbox*) FIFO local con un identificador único UUID y cabecera `Idempotency-Key` (SHA-256) para su despacho automático al detectar reconexión.
- **CU11: Resolver Conflicto de Concurrencia de Stock (`stock = 1`):** Permite detectar y resolver discrepancias cuando dos transacciones offline compiten por la última unidad física en percha. El servidor actúa como autoridad única rechazando la segunda operación con error HTTP 409 (`STOCK_AGOTADO_CONFLICTO`), ante lo cual el cliente ejecuta una transacción compensatoria (rollback local en outbox) y despliega el modal interactivo `ModalConflictoStock` sugiriendo tallas o sucursales alternativas.

Módulo 5: Reportes Generativos con IA y Machine Learning

- **CU12: Monitorear Consola Ejecutiva de Negocio (ADMIN):** Permite al Dueño de boutique auditar en tiempo real la salud financiera de su empresa mediante tarjetas de KPI con deltas simétricos, ticket promedio, rotación de percha (`ventas / stock`), tasa de conversión probador a venta, gráficos Recharts con días rellenados a 0, cuota de pasarelas y consumo de cuotas del plan SaaS.
- **CU13: Operar Consola de Mostrador POS del Día (Cajero):** Permite al Cajero y Encargado gestionar la operativa física de sucursal con corte horario en `America/La_Paz`, visualizando cobranzas pendientes en caja, alertas preventivas de existencias en stock crítico (≤ 2 unidades) o agotadas, y la agenda cronológica de citas del día con recepción de clientas en un solo clic.
- **CU14: Consultar Analítica en Lenguaje Natural e Híbrido con IA:** Permite formular preguntas abiertas de negocio mediante texto o voz. El agente basado en DeepSeek LLM extrae intenciones analíticas hacia un catálogo de 23 métricas parametrizadas (sin generar SQL directo), ejecuta simulaciones *What-If* en memoria con DuckDB y redacta conclusiones gerenciales sin tecnicismos.
- **CU15: Construir Reportes Dinámicos y Exportar a Excel:** Permite al Administrador diseñar informes visuales sin costo de tokens de LLM mediante `ConstructorReporteDialog`, repivotear tablas dinámicas interactivas por dimensiones cruzadas (`sucursal`, `categoria`, `metodo_pago`, `proveedor`) y exportar li-

bros de balance corporativos a formato nativo `.xlsx` (`exceljs`) con formato contable `"Bs" #,##0.00` y fórmulas de totales.

- **CU16: Predecir Demanda de Inventario y Riesgo de Citas (Modelos Supervisados):** Conecta con el microservicio Python FastAPI (:8002) para ejecutar el modelo M1 (`RandomForestRegressor`, $R^2 = 0.9566$) para proyectar la reposición óptima de existencias a 30 días, y el modelo M2 (`LogisticRegression` L2, ROC-AUC 0.7978) para estimar la probabilidad de inasistencia (*no-show*) de citas de probador mediante un semáforo visual.
- **CU17: Auditar Anomalías POS y Segmentar Clientes (Modelos No Supervisados):** Ejecuta el modelo M3 (`KMeans` con $K=4$, Silueta 0.6823) para agrupar clientas según su comportamiento RFM (Recencia, Frecuencia, Valor Monetario), el modelo M4 (`IsolationForest`, contaminación 0.03) para detectar transacciones atípicas o posibles fraudes en caja POS, y permite consultar la Ficha Metodológica reproducible ($N=10.000$, `seed=42`).

2.3. Requisitos Funcionales y No Funcionales

2.3.1. Requisitos Funcionales (RF01 a RF26)

Módulo 1: Gestión Multitenant y Suscripciones (SaaS)

- **RF01 - Gestión de empresas inquilinas (Tenants):** El sistema debe permitir al SuperAdmin registrar, consultar, editar, suspender y reactivar boutiques independientes, aprovisionando en una sola transacción la tienda, el inquilino con su `tenant_id`, su suscripción inicial en período de prueba (`TRIAL`), la sucursal matriz y el usuario `ADMIN`.
- **RF02 - Gestión de planes comerciales de suscripción:** El sistema debe administrar los planes Básico (Bs 99 / 1 sucursal / 50 prendas / 3 staff), Crecimiento (Bs 249 / 3 sucursales / 300 prendas / 10 staff / destacado) y Cadena (Bs 599 / recursos ilimitados), asignando cuotas de recursos y el nivel de marca permitido (`ESENCIAL`, `COMPLETA` o `MARCA_BLANCA`).
- **RF03 - Control y restricción de cuotas de suscripción:** El sistema debe verificar en tiempo de ejecución (`assertDentroDelPlan`) que la tienda no supere las cuotas de su plan al crear o reactivar sucursales, prendas o usuarios staff, rechazando las operaciones excedentes con código HTTP 409 `LimitePlanExcedidoException`.
- **RF04 - Cobro y renovación automatizada de planes (B2B):** El sistema debe permitir al Admin de la boutique contratar, pagar o renovar su plan mensual mediante pasarelas de pago digitales (Stripe Checkout y Wayra QR), persistiendo el registro de cobranza en la tabla `cobros_suscripcion` y actualizando de inmediato la vigencia de la suscripción.
- **RF05 - Aislamiento lógico multi-tenant:** El backend debe inyectar obligatoriamente el contexto de inquilino (`X-Tenant` o JWT) en cada petición mediante `TenantMiddleware` y `TenantInterceptor`, filtrando automáticamente las 22 tablas de negocio con la cláusula `where: soloTenant({...})`, retornando HTTP 404 si se intenta acceder a un recurso de otra tienda.

Módulo 2: Catálogo, Inventario y Puntos de Venta (POS)

- **RF06 - Gestión multivariante de indumentaria femenina:** El sistema debe permitir al Admin registrar prendas clasificadas por categorías, temporadas y colecciones, generando matrices de variantes SKU únicas por combinación de talla y color HEX con precio tipificado en centavos (`precioCents`).
- **RF07 - Control de existencias atómico por sucursal:** El sistema debe controlar las cantidades física (`cantidad_disponible`), reservada (`cantidad_reservada`) y en tránsito (`cantidad_transito`) en cada tienda física, ejecutando las rebajas mediante sentencias SQL `UPDATE` condicionadas con bloqueo pesimista (`pessimistic_write`) para evitar saldos negativos.
- **RF08 - Registro de ventas omnicanal (POS y Digital):** El sistema debe procesar ventas digitales en estado `PENDIENTE` que descuentan stock libre en el checkout y ventas presenciales de mostrador en estado `PAGADA`, generando un código de comprobante determinista institucional `C-XXXXXXXX` derivado del UUID.
- **RF09 - Gestión de abastecimiento con confeccionistas:** El sistema debe permitir emitir órdenes de compra a proveedores textiles aumentando el stock en tránsito de la sucursal de destino, y recepcionar mercancía de forma total o parcial registrando la entrada en kardex.
- **RF10 - Cancelación de ventas con restitución de inventario:** El sistema debe permitir anular ventas no confirmadas o expiradas, liberando de forma inmediata las unidades reservadas, restituyendo el stock libre y devolviendo los artículos al carrito de la clienta.

Módulo 3: Experiencia del Cliente y Probador Virtual por IA

- **RF11 - Personalización estética dinámica (Theming):** La tienda web debe inyectar dinámicamente los tokens de marca del inquilino (logo, fuentes, radios y colores) sobre el DOM, validando que el color principal cumpla un ratio de contraste accesible **WCAG 2.1** de al menos **4.5:1** contra el fondo (`#FFFFFF` o `#0A0A0A`).
- **RF12 - Probador virtual generativo (Decart AI VTON):** La plataforma debe permitir a la clienta probarse prendas virtualmente sobre su foto en tiempo real mediante el motor Decart (`lucy-vton-3.5`), emitiendo tokens efímeros de acceso de 5 minutos protegidos por limitación de tasa por IP (10 peticiones/10 min).
- **RF13 - Consulta de existencias por sucursal:** La ficha de producto debe exhibir en tiempo real el desglose de existencias físicas disponibles en cada sucursal de la cadena para la variante de talla y color seleccionada.
- **RF14 - Gestión de bolsa de compras con topes:** El sistema debe persistir el carrito de la clienta en formato `jsonb`, aplicando una restricción de máximo 3 unidades por variante SKU y un límite de hasta 2 órdenes web pendientes sin pagar simultáneas.
- **RF15 - Sistema de citas para probador físico:** El sistema debe permitir apartar hasta 3 prendas para prueba en sucursal en una ventana temporal entre +30 min y +48 h, bloqueando hasta el 50% del stock libre en `cantidad_reservada` y penalizando con 30 días de suspensión a usuarias con 2 inasistencias en el mes.
- **RF16 - Aplicación y validación de cupones de descuento:** El sistema debe validar cupones de rebaja porcentual o fija con límites de uso y compra mínima, protegiendo el endpoint público contra fuerza bruta mediante limitación de tasa (30 peticiones/5 min).

Módulo 4: Operación Offline-First y Sincronización

- **RF17 - Consulta desconectada del catálogo:** La aplicación web PWA y la aplicación móvil deben permitir navegar el catálogo y consultar existencias cacheadas previamente en IndexedDB (Web) o SQLite (Móvil) ante pérdida total de conexión a Internet, preservando los datos por 7 días (TTL).
- **RF18 - Ventas y compras sin conexión:** El cajero debe poder registrar ventas de contado en el Mostrador sin conexión, y la clienta comprar desde la app móvil sin conexión; cada operación lleva un UUIDv4 como `Idempotency-Key` y se encola localmente hasta reconectar. Las citas de probador (reservas) requieren conexión.
- **RF19 - Registro de ventas presenciales de mostrador offline:** El sistema debe permitir al cajero procesar cobros de contado en efectivo sin conexión desde el POS de la PWA, descontando el inventario local y almacenando la transacción en la cola *outbox*.
- **RF20 - Cola de salida (*outbox*) con monitoreo de estados:** El cliente debe proveer una estructura de cola local que rastree el ciclo de vida de cada operación offline (`PENDIENTE`, `SINCRONIZANDO`, `SINCRONIZADO`, `COMPENSADO` o `FALLIDO`), permitiendo la depuración de órdenes confirmadas.
- **RF21 - Sincronización asíncrona e idempotente:** Al restablecerse la conectividad a Internet, el sistema debe drenar automáticamente la cola outbox hacia el backend enviando la cabecera `Idempotency-Key` (SHA-256), garantizando que reintentos sucesivos no dupliquen transacciones ni cobros (`X-Cache-Idempotency: HIT`).
- **RF22 - Resolución de conflicto concurrente `stock = 1`:** El backend central debe validar la disponibilidad física al procesar operaciones sincronizadas; si la prenda ya fue adquirida por otro canal, debe responder HTTP 409 (`STOCK_AGOTADO_CONFLICTO`), activando en el cliente una transacción compensatoria de rollback local y sugiriendo alternativas inmediatas.

Módulo 5: Reportes Generativos con IA y Machine Learning

- **RF23 - Consultas analíticas en lenguaje natural (Texto / Voz):** El sistema debe permitir al Administrador formular solicitudes de información en lenguaje natural mediante teclado o dictado por voz, mapeándolas mediante DeepSeek LLM hacia un catálogo estricto de 23 métricas parametrizadas sin generar sentencias SQL directas.
- **RF24 - Simulación analítica *What-If* y exportación a Excel:** El sistema debe permitir ejecutar simulaciones predictivas en memoria mediante el motor analítico DuckDB (`:memory:`) y exportar tablas de datos consolidadas hacia libros nativos de Excel `.xlsx` (`exceljs`) con formato contable `"Bs" #,##0.00` y fórmulas GAAP.
- **RF25 - Tableros de control de Inteligencia de Negocios (BI) por rol:** El sistema debe ofrecer la Consola Ejecutiva para el `ADMIN` con indicadores de facturación, deltas simétricos, rotación y conversión, y la Consola Operativa POS para el `CAJERO` / `ENCARGADO` con agenda cronológica de citas y alertas de stock crítico ≤ 2 unidades.
- **RF26 - Inferencia predictiva mediante Machine Learning (:8002):** El sistema debe integrar un microservicio desacoplado en Python FastAPI que procese 4 modelos entrenados reproducibles ($N=10.000$, `seed=42`): M1 (Demanda Random Forest), M2 (No-Show Regresión Logística), M3 (Segmentación RFM K-Means) y M4 (Detección de Anomalías Isolation Forest), con fallback heurístico resiliente (`X-ML-Fallback: true`).

2.3.2. Requisitos No Funcionales (RNF01 a RNF12)

- **RNF01. Seguridad Criptográfica y RBAC:** La autenticación se basa en JSON Web Tokens (JWT) con algoritmo HMAC-SHA256 y vigencia de 1 hora, revalidados obligatoriamente contra la base de datos en cada petición. Las contraseñas se almacenan mediante funciones de derivación de claves seguras (**BCrypt** con factor de coste ≥ 10) y se aplican limitadores de tasa por IP en memoria para prevenir ataques de fuerza bruta en login (10 peticiones/5 min).
- **RNF02. Aislamiento Lógico Multi-Tenant:** Ninguna operación debe permitir consultar, alterar o eliminar información perteneciente a otro inquilino. La separación se aplica de forma forzosa en la capa de servicios TypeORM mediante la cláusula `where: soloTenant({...})` e inyección de `tenant_id` en todas las consultas SQL, respondiendo invariablemente HTTP 404 ante identificadores de otra tienda para prevenir fuga de metadatos.
- **RNF03. Persistencia Desconectada Resiliente:** Las operaciones registradas sin conexión deben almacenarse en motores estructurados locales (**IndexedDB** en la PWA web y base relacional embebida **SQLite** en la app móvil Expo), sobreviviendo intactas al cierre de pestaña, recarga o reinicio del sistema operativo.
- **RNF04. Integridad Transaccional ACID:** Toda transacción que afecte el inventario o procese un cobro debe ejecutarse dentro de transacciones atómicas gestionadas por PostgreSQL 16 con bloqueos pesimistas (`SELECT ... FOR UPDATE`), garantizando que jamás se produzcan descuadres entre existencias y ventas confirmadas.
- **RNF05. Idempotencia Rigurosa:** El reenvío sucesivo de una orden diferida (por fallas transitorias de red o reconexiones múltiples) identificado con la misma cabecera `Idempotency-Key` no debe generar duplicidad de compras ni dobles rebajas de inventario, retornando la respuesta original en caché (`X-Cache-Idempotency: HIT`).
- **RNF06. Recuperación ante Fallos y Transacción Compensatoria:** Ante respuestas de rechazo por parte del servidor central (como HTTP 409 por stock agotado), el cliente no debe purgar la orden de forma silenciosa, sino transicionarla al estado `COMPENSADO`, ejecutar el rollback local y notificar de manera interactiva a la usuaria.
- **RNF07. Modularidad y Desacoplamiento:** La arquitectura backend debe estructurarse como un Monolito Modular donde los módulos de negocio no importan entidades de otros módulos, comunicándose exclusivamente mediante servicios exportados e identificadores foráneos planos `UUID`, permitiendo que el microservicio de ML (`mirroria-ml`) opere en un proceso autónomo en puerto `8002`.
- **RNF08. Compatibilidad y Soporte Omnicanal:** La aplicación web React 19 debe cumplir los estándares de Progressive Web App (PWA) con Service Worker y Web Manifest para operar en navegadores de escritorio y móviles; el cliente móvil en React Native / Expo SDK 57 debe garantizar portabilidad multiplataforma sobre dispositivos Android e iOS.
- **RNF09. Rendimiento y Tiempo de Respuesta:** Las consultas transaccionales de catálogo y pedidos deben responder en un tiempo inferior a **200 ms** bajo condiciones normales, mientras que las inferencias analíticas de Machine Learning y consultas a DuckDB no deben exceder un tiempo límite de **1.500 ms**, activando el fallback heurístico en caso de sobrepasar este umbral.
- **RNF10. Observabilidad y Trazabilidad:** Todo error de negocio debe emitir una estructura homogénea `{ status, message, timestamp }` evitando fugas de trazas internas del servidor. La telemetría de inferencia debe registrar la cabecera `X-ML-Fallback: true/false` para auditar la salud del microservicio Python.

- **RNF11. Calidad del Software y Cobertura de Pruebas:** El sistema debe contar con suites de pruebas automatizadas exhaustivas que certifiquen el funcionamiento del software: 557 pruebas unitarias y 61 de integración en backend (Vitest), 345 pruebas en frontend, 17 en la app móvil, y scripts de QA de caja negra que certifiquen el aislamiento multi-tenant y reglas de mostrador.
- **RNF12. Accesibilidad Universal WCAG 2.1:** Todo tema o paleta visual personalizada por una boutique debe ser validada algorítmicamente asegurando un ratio de contraste mínimo de **4.5:1** (Nivel AA) entre los colores de texto y los fondos preestablecidos (`#FFFFFF` o `#0A0A0A`).

2.4. Reglas de Negocio

Categoría 1: Gobernanza Multitenant y Suscripciones

- **RN01 - Aislamiento Estricto por Tenant:** Toda entidad de negocio (usuarios, sucursales, productos, variantes, clientes, reservas, ventas, pagos y movimientos de inventario) pertenece de forma obligatoria a una tienda (`tenant_id NOT NULL`). Ningún usuario puede operar fuera del alcance de su inquilino. Cualquier petición con JWT de una tienda que intente operar sobre otra recibe un error HTTP 403 `"Tu sesión pertenece a otra tienda"` , y un recurso con ID inexistente en la tienda solicitada responde HTTP 404 `"No encontrado"` .
- **RN02 - Límites de Recursos según Plan Comercial:** El número de sucursales físicas, prendas maestras activas y usuarios de staff de una boutique no puede superar las cuotas contratadas en su plan vigente:
- **Plan Básico (Bs 99):** Máximo 1 sucursal, 50 prendas activas y 3 usuarios de staff. Nivel de marca `ESENCIAL` .
- **Plan Crecimiento (Bs 249):** Máximo 3 sucursales, 300 prendas activas y 10 usuarios de staff. Nivel de marca `COMPLETA` .
- **Plan Cadena (Bs 599):** Recursos ilimitados. Nivel de marca `MARCA_BLANCA` (permite favicon propio y remover insignia). El backend valida estas restricciones en tiempo de ejecución (`assertDentroDelPlan`) al invocar `POST /sucursales` , `POST /catalogo/productos` o al crear usuarios staff, rechazando transacciones excedentes con HTTP 409 (`LimitePlanExcedidoException`).
- **RN03 - Vigencia y Suspensión de la Suscripción:** Una tienda cuya suscripción se encuentre en estado vencida o suspendida por mora tiene bloqueadas todas las operaciones mutantes de escritura (`POST` , `PUT` , `PATCH` , `DELETE`), respondiendo HTTP 403 `"Tu tienda se encuentra suspendida o con suscripción vencida"` . Únicamente se permite la consulta de información y la navegación a `/admin/suscripcion/activar` para regularizar el pago.
- **RN04 - Confinamiento y Accesibilidad de Marca (WCAG 2.1):** Ninguna boutique puede publicar una combinación de color principal cuyo contraste calculado sobre el fondo de la tienda (`#FFFFFF` en modo claro y `#0A0A0A` en modo oscuro) sea inferior al ratio de **4.5:1**. El sistema rechaza colores no conformes con error HTTP 400 y ofrece en la interfaz una herramienta de ajuste perceptual automático en espacio cromático OKLCH.

Categoría 2: Operaciones Comerciales e Inventario

- **RN05 - Descuento Atómico y Existencias No Negativas:** Toda venta web o presencial debe verificar el stock libre disponible y ejecutar la deducción de unidades de manera atómica mediante sentencias SQL

condicionadas (`UPDATE inventario_sucursal SET cantidad_disponible = cantidad_disponible - :cant WHERE cantidad_disponible >= :cant`) dentro de transacciones aisladas con bloqueo pesimista. Las existencias jamás pueden asumir valores negativos.

- **RN06 - Retención y Reglas Anti-Acaparamiento en Reservas:** Una reserva de prendas para probarse en sucursal retiene existencias en la columna `cantidad_reservada` bajo estrictas restricciones:
- Máximo 3 prendas por reserva y 1 unidad por variante.
- Máximo 2 reservas activas simultáneas por clienta.
- La cita debe agendarse con una anticipación mínima de 30 minutos y máxima de 48 horas.
- La fracción máxima bloqueable no puede superar el 50% de las existencias libres de la sucursal.
- Si una clienta acumula **2 inasistencias** (`NO_SHOW` o `EXPIRADA`) en un período de 30 días, el sistema suspende automáticamente su derecho a reservar citas durante **30 días**.
- **RN07 - Restricciones del Carrito y Ventas Pendientes:** Una clienta puede agregar como máximo 3 unidades de una misma variante SKU a su carrito de compras digital, y no puede mantener más de **2 órdenes de compra web simultáneas en estado** `PENDIENTE` sin pagar, previniendo el bloqueo abusivo de inventario en percha.

Categoría 3: Operación Offline y Protocolo de Sincronización

- **RN08 - Restricción de Medios de Pago en Modo Desconectado:** Las ventas presenciales procesadas sin conexión a Internet desde el POS de la PWA se registran exclusivamente como transacciones de **contado en efectivo**; los cobros con tarjeta o código QR dinámico exigen forzosamente conectividad activa con las pasarelas bancarias.
- **RN09 - Autoridad Central en Conciliación de Existencias:** La confirmación de una venta o reserva en almacenamiento local (IndexedDB o SQLite) no constituye una aceptación definitiva. La validez jurídica y el asiento definitivo de inventario quedan supeditados a la conciliación en el servidor central al restablecerse la red.
- **RN10 - Persistencia Inviolable de la Cola Outbox:** Ninguna transacción diferida puede ser eliminada del almacenamiento local por fallas transitorias de red o rechazos de sincronización. Las órdenes permanecen en estado `PENDIENTE` o `FALLIDO` hasta que sean resueltas o depuradas voluntariamente por el operador de caja.
- **RN11 - Idempotencia Obligatoria en Sincronización:** Toda petición mutante sincronizada debe incorporar la cabecera `Idempotency-Key` con un identificador UUIDv4 y el hash SHA-256 canónico del payload. Reenvíos sucesivos de una misma orden confirmada retornan la respuesta almacenada en la tabla `idempotencia` (`X-Cache-Idempotency: HIT`) sin duplicar ventas ni movimientos de stock.
- **RN12 - Tratamiento del Conflicto de Concurrencia** `stock = 1`**:** Si al sincronizar una venta offline el stock en percha ya fue adquirido por otro canal, el servidor emite una respuesta HTTP 409 tipada (`STOCK_AGOTADO_CONFLICTO`). El cliente intercepta este código, transiciona la orden a `COMPENSADO` (rollback local), revierte la deducción provisional y despliega el modal interactivo `ModalConflictoStock` sugiriendo sucursales o tallas sustitutas sin bloquear al usuario.

Categoría 4: Inteligencia Artificial y Analítica

- **RN13 - Confinamiento y Seguridad en Consultas de IA:** El motor conversacional asistido por DeepSeek LLM tiene **estrictamente prohibido generar sentencias SQL directas**. Toda solicitud en lenguaje natural debe traducirse hacia una ficha analítica validada que consulta exclusivamente el catálogo formal de 23 métricas parametrizadas, inyectando de forma forzosa el `tenant_id` y sucursal del usuario logueado.
- **RN14 - Veracidad de Datos y Resiliencia en Inferencia ML:** Los reportes generativos y tableros analíticos deben construirse exclusivamente sobre datos reales del inquilino, informando la ausencia de datos en lugar de alucinar valores. En caso de indisponibilidad del microservicio Python en puerto `8002`, el sistema conmuta de inmediato hacia el **fallback heurístico matemático**, alertando la condición mediante la cabecera `X-ML-Fallback: true`.

2.5. Diagrama General UML de Casos de Uso (UML 2.5)

El siguiente diagrama modela las interacciones entre los actores del sistema y los 17 casos de uso consolidados del 2.º Ciclo, estructurados en los 5 módulos funcionales:

2.6. Especificación Detallada de Casos de Uso del 2.º Ciclo

A continuación se presentan las especificaciones formales de los Casos de Uso nucleares del 2.º Ciclo académico, siguiendo la plantilla estructurada de la cátedra de Sistemas de Información II:

CU01: Aprovisionar Tienda y Workspace SaaS

Campo	Valor
Nombre:	Aprovisionar Tienda y Workspace SaaS
Autor:	Equipo de Desarrollo MirrorIA
Fecha:	06/11/2026

Descripción: Permite a un Visitante o futuro Dueño de Boutique registrar su tienda mediante un flujo de autoservicio en línea, seleccionando un plan comercial. El sistema aprovisiona de forma atómica en PostgreSQL la empresa inquilina (`inquilinos`), asigna el identificador global `tenant_id`, activa un período de prueba de cortesía (`TRIAL` de 14 días), crea el usuario inicial con rol `ADMIN`, registra la sucursal matriz física de atención, crea el proveedor "Producción propia" y siembra un catálogo base de prendas, garantizando el aislamiento lógico de datos y la navegabilidad en `/t/:slug`.

Actores: Visitante, Dueño de Boutique (`ADMIN`), Pasarela de Pagos (Stripe / Wayra QR).

Pre-Condiciones: 1. El usuario tiene acceso a la landing corporativa de MirrorIA (`/planes` o `/registro-tienda`). 2. Existen planes comerciales activos y configurados en la base de datos (Básico, Crecimiento o Cadena).

Flujo Normal: 1. El visitante accede a `/planes` y evalúa las características y límites de los planes comerciales. 2. El visitante selecciona un plan (ej. *Crecimiento*) y pulsa en "Comenzar prueba gratis de 14 días". 3. El sistema muestra el formulario de alta de boutique (`/registro-tienda`). 4. El usuario ingresa el nombre comercial de su tienda y el slug deseado para la URL (ej. `tienda-boutique`). 5. El sistema consulta en tiempo real mediante `GET /saas/tiendas/disponibilidad?slug=...` y confirma que el identificador está disponible. 6. El usuario selecciona la ciudad capital de su sucursal matriz y digita la dirección física. 7. El usuario ingresa los datos del Administrador: nombre completo, correo electrónico y contraseña maestra (mínimo 8 caracteres). 8. El usuario envía el formulario pulsando en "Crear mi tienda". 9. El backend valida los campos mediante `ValidationPipe` y ejecuta una transacción atómica que: - Inserta la fila en la tabla `inquilinos` con estado `TRIAL` y fecha de vencimiento a +14 días. - Crea el registro en `inquilino_marca` con la paleta de colores neutra por defecto. - Crea la suscripción vinculada en la tabla `suscripciones`. - Hashea la contraseña con `BCrypt` y crea el usuario en `usuarios` con rol `ADMIN` y `tenant_id` fijado. - Inserta la primera sucursal en `sucursales` vinculada a la ciudad elegida. - Crea el proveedor inicial "Producción propia" en `proveedores`. - Siembra categorías y prendas demo mediante `CatalogoBaseService.sembrarBase`. 10. El sistema emite un token JWT de sesión autenticado, almacena las credenciales y redirige a la dueña directamente al panel de control `/admin/dashboard` de su tienda.

Flujos Alternativos: * **4a. Slug reservado o en uso:** Si el slug coincide con una palabra reservada del sistema (`admin`, `plataforma`, `api`) o ya pertenece a otra tienda, el sistema emite HTTP 400 o 409 indicando `"El nombre de tienda ya se encuentra en uso o no está disponible."`. * **7a. Contraseña vulnerable o correo inválido:** Si la contraseña tiene menos de 8 caracteres o el formato de correo es incorrecto, el sistema rechaza el alta con HTTP 400 y resalta los campos con error. * **9a. Falla transaccional en base de datos:** Ante una colisión de clave única (código Postgres `23505`), el backend ejecuta un rollback completo de la transacción, garantizando que no queden datos huérfanos, y responde con código `409 Conflict`.

Poscondiciones: La tienda queda completamente operativa e indexada con su `tenant_id` único, sucursal base y catálogo inicial; el usuario `ADMIN` queda logueado con token JWT y habilitado para operar su boutique en `/admin/*` y `/t/:slug`.

CU03: Personalizar Identidad Visual de Boutique

Campo	Valor
Nombre:	Personalizar Identidad Visual de Boutique
Autor:	Equipo de Desarrollo MirrorIA
Fecha:	06/11/2026

Descripción: Permite al Administrador de la boutique (`ADMIN`) personalizar la identidad visual de su tienda web pública (`/t/:slug`), configurando logotipo, favicon, color principal, color de acento, tipografía de títulos, radio de bordes, modo visual (`CLARO` u `OSCURO`) e imágenes de cabecera, validando de forma algorítmica y obligatoria que los contrastes cromáticos satisfagan las directrices internacionales de accesibilidad **WCAG 2.1**.

Actores: Dueño de Boutique (`ADMIN`), SuperAdmin (`DUENO_PLATAFORMA` para restablecimiento de oficio).

Pre-Condiciones: 1. El Administrador ha iniciado sesión con un token JWT válido perteneciente a su boutique. 2. El plan de suscripción de la boutique se encuentra activo o en prueba (`TRIAL`).

Flujo Normal: 1. El Admin accede al menú lateral y pulsa en "Mi negocio" $\to$ "Apariencia" (`/admin/apariencia`). 2. El sistema consulta mediante `GET /api/v1/saas/mi-marca` y carga la configuración actual de marca junto con el nivel de personalización permitido por su plan (`ESENCIAL`, `COMPLETA` o `MARCA_BLANCA`). 3. El Admin modifica el color principal y el color de acento ingresando códigos HEX o seleccionando una paleta sugerida. 4. El sistema ejecuta en tiempo real la función matemática de luminancia relativa y contraste WCAG 2.1: $$ \text{Ratio} = \frac{L_1 + 0.05}{L_2 + 0.05}$$ contrastando el color principal contra el fondo de la tienda (`#FFFFFF` en modo claro y `#0A0A0A` en modo oscuro). 5. El sistema confirma que el ratio de contraste es mayor o igual a **4.5:1** (Nivel AA) y marca el indicador en estado verde accesible. 6. El Admin selecciona el estilo tipográfico de encabezados (Outfit, DM Sans o Playfair) y el radio de bordes de los botones. 7. El Admin pulsa en "Guardar cambios". 8. El backend valida en `marca-reglas.ts` que los campos enviados no violen las restricciones del plan de suscripción (`nivelMarca`), persiste los cambios en la tabla `inquilino_marca` y responde HTTP 200. 9. El sistema inyecta en caliente las nuevas variables CSS sobre el DOM y actualiza la caché local de marca (`localStorage`).

Flujos Alternativos: * **4a. Contraste insuficiente (WCAG < 4.5:1):** Si el ratio de contraste es inferior a 4.5:1, la interfaz despliega una alerta ámbar de accesibilidad (`AvisoContraste`) y deshabilita el guardado. Al pulsar "Ajustar contraste", el sistema ejecuta la función `ajustarContraste` que recalcula perceptualmente la luminosidad en espacio OKLCH hasta alcanzar el umbral de 4.5:1 automáticamente. * **6a. Intento de uso de características no incluidas en el plan:** Si una tienda en plan Básico (`nivelMarca: ESENCIAL`) intenta habilitar el modo oscuro o subir favicon propio, el backend rechaza la petición con HTTP 409 (`LimitePlanExcedidoException`), informando que se requiere el plan Crecimiento o Cadena. * **8a. Restablecimiento de oficio:** Si la tienda contiene imágenes o colores inapropiados, el SuperAdmin puede invocar `POST /saas/plataforma/inquilinos/:id/marca/restablecer` para restaurar la paleta neutra de fábrica.

Poscondiciones: La tienda pública `/t/:slug` adopta inmediatamente la nueva paleta de colores, tipografía y logotipo en todas las vitrinas y fichas de producto sin afectar a otros inquilinos.

CU10: Operar en Modo Desconectado con Sincronización Segura

Campo	Valor
Nombre:	Operar en Modo Desconectado con Sincronización Segura
Autor:	Equipo de Desarrollo MirrorIA
Fecha:	06/11/2026

Descripción: Permite a la Clienta y al Cajero continuar utilizando la plataforma ante caídas imprevistas de conexión a Internet, navegando catálogos guardados y registrando ventas presenciales de contado en efectivo (Mostrador) o compras desde la app móvil desde la PWA (IndexedDB) o la app móvil (SQLite). Las operaciones se encolan con identificador UUIDv4 y se transmiten automáticamente de forma idempotente con hash SHA-256 al recuperar la red.

Actores: Clienta (`CUSTOMER`), Cajero (`CAJERO`), Service Worker, Backend NestJS.

Pre-Condiciones: 1. El usuario abrió la aplicación previamente con conexión a Internet, permitiendo al Service Worker cachear el App Shell y descargar el catálogo local con existencias en IndexedDB o SQLite. 2. El dispositivo sufre una pérdida total o parcial de conectividad a la red.

Flujo Normal: 1. El usuario navega la tienda o el POS; el sistema detecta el evento de desconexión y activa la barra visual de "Modo Offline". 2. El sistema carga las prendas y precios directamente desde el almacén `catalogo_cache` de IndexedDB (Web) o base local `SQLite` (Móvil). 3. El cajero en `/admin/mostrador` selecciona prendas y cantidades y pulsa "Cobrar en efectivo". 4. El cliente verifica las existencias locales disponibles y deduce provisionalmente el stock local. 5. El cliente genera un identificador UUIDv4 como clave de idempotencia y anota la hora del cobro (`vendidaEn`), con la que el servidor valida el precio cobrado. 6. La transacción se encola en el almacén local `outbox` con estado `PENDIENTE` junto con la cabecera `Idempotency-Key` (UUIDv4) y el hash SHA-256 del cuerpo de la petición. 7. El sistema confirma visualmente que la venta fue guardada localmente y deja el ticket disponible para la siguiente clienta. 8. Al restablecerse la conexión a Internet, el Service Worker (o el listener de NetInfo en móvil) detecta el evento `online` y activa el vaciado de la cola FIFO (`outbox-manager.ts`). 9. El cliente despacha la venta mediante `POST /api/v1/ventas/presenciales` (en el móvil, `POST /api/v1/ventas/carrito/:id/checkout` con sus prendas) enviando la cabecera `Idempotency-Key`. 10. El interceptor global de idempotencia en el backend verifica que la clave no haya sido procesada previamente y ejecuta la transacción en PostgreSQL. 11. El backend responde HTTP 201 (`X-Cache-Idempotency: MISS`) con el comprobante definitivo `C-XXXXXXXX`. 12. El cliente actualiza la orden en la cola outbox pasando su estado a `SINCRONIZADO` y actualiza la caché local de catálogo con las existencias confirmadas del servidor.

Flujos Alternativos: * **8a. Falla de red transitoria durante el envío:** Si la conexión cae a mitad de la transmisión, la orden regresa al estado `PENDIENTE` y se reprograma el reintento automático sin pérdida de datos. * **10a. Petición duplicada (Reenvío concurrente):** Si la orden ya había ingresado al servidor pero la respuesta HTTP se perdió en el aire, el reenvío encuentra la llave en la tabla `idempotencia` y retorna inmediatamente la respuesta original cacheada HTTP 200 (`X-Cache-Idempotency: HIT`), garantizando cero doble cobro o doble descuento de stock.

Poscondiciones: La venta queda asentada formalmente en la base de datos central de PostgreSQL con su respectivo comprobante fiscal y movimiento de kardex; la cola outbox local queda limpia y el inventario sincronizado.

CU11: Resolver Conflicto de Concurrencia de Stock (`stock = 1`)

Campo	Valor
Nombre:	Resolver Conflicto de Concurrencia de Stock (`stock = 1`)
Autor:	Equipo de Desarrollo MirrorIA
Fecha:	06/11/2026

Descripción: Permite detectar, aislar y resolver la colisión de inventario que ocurre cuando dos transacciones registradas de forma concurrente o desconectada compiten por la última prenda disponible en percha (`stock = 1`). El servidor central actúa como autoridad transaccional única rechazando la segunda operación, ante lo cual el cliente ejecuta una transacción compensatoria (rollback) y sugiere alternativas inmediatas.

Actores: Clienta (`CUSTOMER`), Cajero (`CAJERO`), Backend NestJS.

Pre-Condiciones: 1. Existe una sola unidad disponible (`cantidad_disponible = 1`) de una variante de prenda en una sucursal determinada. 2. Dos clientas (o un cajero offline y una compradora web) intentan adquirir la misma prenda en paralelo.

Flujo Normal: 1. La primera transacción llega al servidor central; el servicio `InventarioSucursalService` adquiere un bloqueo pesimista exclusivo sobre la fila (`SELECT ... FOR UPDATE`). 2. El servidor constata que `cantidad_disponible = 1 >= 1`, descuenta la unidad a 0, asienta la venta y confirma la transacción exitosamente. 3. La segunda transacción (originada sin conexión o despachada milisegundos después) es procesada por el servidor con su correspondiente `Idempotency-Key`. 4. El servidor ejecuta la sentencia de deducción atómica `UPDATE inventario_sucursal SET cantidad_disponible = cantidad_disponible - 1 WHERE cantidad_disponible >= 1`. 5. Al haber 0 unidades disponibles, la consulta afecta 0 filas; el servidor detecta el quiebre de stock y dispara la excepción tipada `StockInsuficienteException`. 6. El backend interrumpe la transacción y responde con código HTTP **409 Conflict**, retornando la estructura tipada: `json { "status": 409, "codigo": "STOCK_AGOTADO_CONFLICTO", "message": "Stock insuficiente para la variante solicitada", "detalles": { "varianteId": "...", "sucursalId": "...", "disponible": 0, "solicitado": 1, "alternativasSucursales": [...], "alternativasTallas": [...] } }` 7. El gestor de cola local del cliente (`outbox-manager.ts`) detecta el código `STOCK_AGOTADO_CONFLICTO`, transiciona la orden a estado **`COMPENSADO`** y ejecuta la **transacción compensatoria** restituyendo provisionalmente las existencias en el dispositivo. 8. El sistema dispara el evento global `mirroria:conflicto-stock` abriendo en pantalla el diálogo interactivo `ModalConflictoStock`. 9. La clienta (o el cajero) visualiza la notificación: *"La prenda seleccionada acaba de ser adquirida por otra clienta. Tu venta local fue cancelada sin cobro."*. 10. El modal ofrece dos alternativas directas: - Retirar la prenda en otra sucursal cercana con existencias. - Seleccionar otra talla disponible del mismo modelo. 11. La clienta selecciona una alternativa sugerida y el sistema reinicia el proceso de compra de forma fluida.

Flujos Alternativos: * **10a. Descarte de la orden:** Si la clienta no desea ninguna de las alternativas sugeridas, pulsa en "Cerrar y descartar"; el sistema purga la orden compensada de la cola outbox y actualiza el carrito.

Poscondiciones: La base de datos central preserva la coherencia física de inventario (cero sobreventa y cero existencias negativas); en el cliente no se produce ningún cargo monetario y la usuaria recibe opciones inmediatas para concretar su compra.

CU12: Monitorear Consola Ejecutiva de Negocio (ADMIN)

Campo	Valor
Nombre:	Monitorear Consola Ejecutiva de Negocio (ADMIN)
Autor:	Equipo de Desarrollo MirrorIA
Fecha:	06/11/2026

Descripción: Permite al Administrador y dueño de la boutique acceder a un tablero directivo de Inteligencia de Negocios (BI) para evaluar en tiempo real la salud financiera de su empresa, analizando ingresos netos del período con deltas simétricos, ticket promedio, rotación de percha, tasa de conversión de probador a venta, distribución de pasarelas de pago y consumo de recursos de su plan SaaS.

Actores: Dueño de Boutique (`ADMIN`), Encargado de Sucursal.

Pre-Condiciones: 1. El usuario ha iniciado sesión con rol `ADMIN` en su boutique. 2. Existen transacciones de venta e inventario registradas en el período consultado.

Flujo Normal: 1. El Admin ingresa a la plataforma web y accede a `/admin/dashboard`. 2. El sistema identifica el rol `ADMIN` y monta la vista ejecutiva (`DashboardEjecutivoView`). 3. El frontend invoca `GET /api/v1/dashboard/ejecutivo` enviando los parámetros de filtrado seleccionados (período `hoy`, `7d`, `30d` o rango personalizado, y sucursal). 4. El backend (`BiService`) ejecuta consultas analíticas consolidadas sobre las tablas `ventas`, `pagos`, `inventario_sucursal` y `reservas`, calculando: - Facturación neta del período y del período anterior simétrico, calculando la variación porcentual. - Ticket promedio de compra (`ingresos_totales / total_ventas`). - Tasa de rotación de inventario ($\text{Rotación} = \text{unidades_vendidas} / \text{stock_disponible}$). - Tasa de conversión de reservas físicas a ventas efectivas en caja. - Curva de tendencia temporal de ingresos con días vacíos rellenados a 0 para el gráfico Recharts. - Desglose porcentual por método de pago (Tarjeta Stripe, QR Wayra y Efectivo). - Top 5 prendas más vendidas y top proveedores por facturación (inmunes a sesgos de fan-out). - Consumo de cuotas del plan SaaS (`UsoPlanService`). 5. El sistema presenta el tablero con tarjetas de KPI estilizadas, gráficos de barras interactivos y barras de progreso del plan. 6. El Admin interactúa con el selector de sucursal para auditar una tienda específica o la cadena completa.

Flujos Alternativos: * **3a. Tienda nueva sin transacciones:** Si la boutique no registra ventas en el período, el sistema despliega el tablero en estado vacío amigable con ceros balanceados, sin arrojar errores de división entre cero ni pantallas rotas.

Poscondiciones: El Administrador obtiene visibilidad analítica integral para la toma de decisiones comerciales sin alterar el estado transaccional del sistema.

CU16: Predecir Demanda de Inventario y Riesgo de Citas (Modelos Supervisados)

Campo	Valor
Nombre:	Predecir Demanda de Inventario y Riesgo de Citas (Modelos Supervisados)
Autor:	Equipo de Desarrollo MirrorIA
Fecha:	06/11/2026

Descripción: Permite al Administrador y al Encargado de Sucursal consultar proyecciones analíticas generadas por modelos supervisados de Machine Learning desacoplados en el microservicio Python FastAPI (:8002): el modelo **M1 (Random Forest Regressor)** para pronosticar la demanda de prendas a 30 días y optimizar pedidos de reposición, y el modelo **M2 (Regresión Logística L2)** para clasificar el riesgo de inasistencia (*no-show*) de citas de probador mediante un semáforo visual en el mostrador.

Actores: Dueño de Boutique (`ADMIN`), Encargado de Sucursal, Microservicio ML (`mirroria-ml`).

Pre-Condiciones: 1. El usuario tiene sesión iniciada con rol `ADMIN` o `ENCARGADO_SUCURSAL`. 2. Existen prendas registradas y citas de probador programadas en la sucursal.

Flujo Normal: 1. El usuario accede a la sección de "Predicciones" (`/admin/ml`) o consulta la agenda de citas en la Consola Operativa POS (`/admin/dashboard`). 2. El frontend solicita las predicciones mediante `GET /api/v1/ml/kpis/demanda?dias=30` y `GET /api/v1/ml/kpis/no-show`. 3. El módulo `MlModule` en NestJS canaliza la petición hacia el microservicio Python en `http://mirroria-ml:8002/predict/demanda` y `/predict/no-show`. 4. El microservicio ejecuta la inferencia estadística sobre los artefactos entrenados (`joblib`): - **M1 (Demanda):** `RandomForestRegressor` ($R^2 = 0.9566$, MAE 1.91) calcula las ventas proyectadas por prenda, calcula los días de cobertura según el stock en percha y recomienda la cantidad exacta a reponer con un margen de seguridad del 20%: $$\text{Sugerido} = \max(0, \lceil \text{Demanda} \times 1.2 \rceil - \text{Stock})$$ - **M2 (No-Show):** `LogisticRegression` L2 (ROC-AUC 0.7978, $F_1 = 0.7440$) estima la probabilidad de inasistencia de cada clienta citada y asigna el semáforo de riesgo: Verde ($< 35\%$), Amarillo ($35-65\%$) o Rojo ($\ge 65\%$). 5. El backend devuelve los resultados inyectando la cabecera `X-ML-Fallback: false`. 6. La interfaz presenta las tarjetas de reposición sugerida en lenguaje directivo y marca en la agenda POS las citas que requieren confirmación prioritaria por WhatsApp.

Flujos Alternativos: * **3a. Microservicio Python apagado o con caída de red:** Si la llamada a `mirroria-ml:8002` supera el tiempo límite de 1.500 ms o falla la conexión, el método `fetchMl` en NestJS activa de inmediato el **fallback heurístico matemático**. Calcula la demanda proporcional al historial y emite las alertas con la cabecera `X-ML-Fallback: true`. La interfaz muestra una etiqueta informativa indicando que los datos corresponden a una estimación matemática de respaldo. * **4a. Consulta de rigor científico:** El usuario pulsa en el botón "¿Cómo se calcula?" en el encabezado; el sistema despliega el panel lateral `FichaTecnicaSheet` exponiendo el tamaño de muestra ($N=10.000$), semilla (`seed=42`), métricas de bondad de ajuste (R^2, MAE, ROC-AUC) y citas académicas formales (Breiman 2001, Cox 1958).

Poscondiciones: El personal de tienda dispone de recomendaciones cuantitativas precisas para ordenar confección a proveedores textiles y mitigar la inmovilización de existencias por inasistencias en probador.

CAPÍTULO III. ANÁLISIS Y DISEÑO CON ICONIX

3.1. Modelo de Dominio

El **Modelo de Dominio** constituye el vocabulario visual y conceptual del sistema MirrorIA, capturando las entidades fundamentales del negocio de moda femenina bajo la arquitectura SaaS Multi-tenant. Para garantizar el aislamiento lógico de datos entre boutiques, todas las entidades transaccionales y de configuración operativa incorporan el discriminador `tenantId: UUID` como llave foránea hacia la entidad central `Inquilino`.

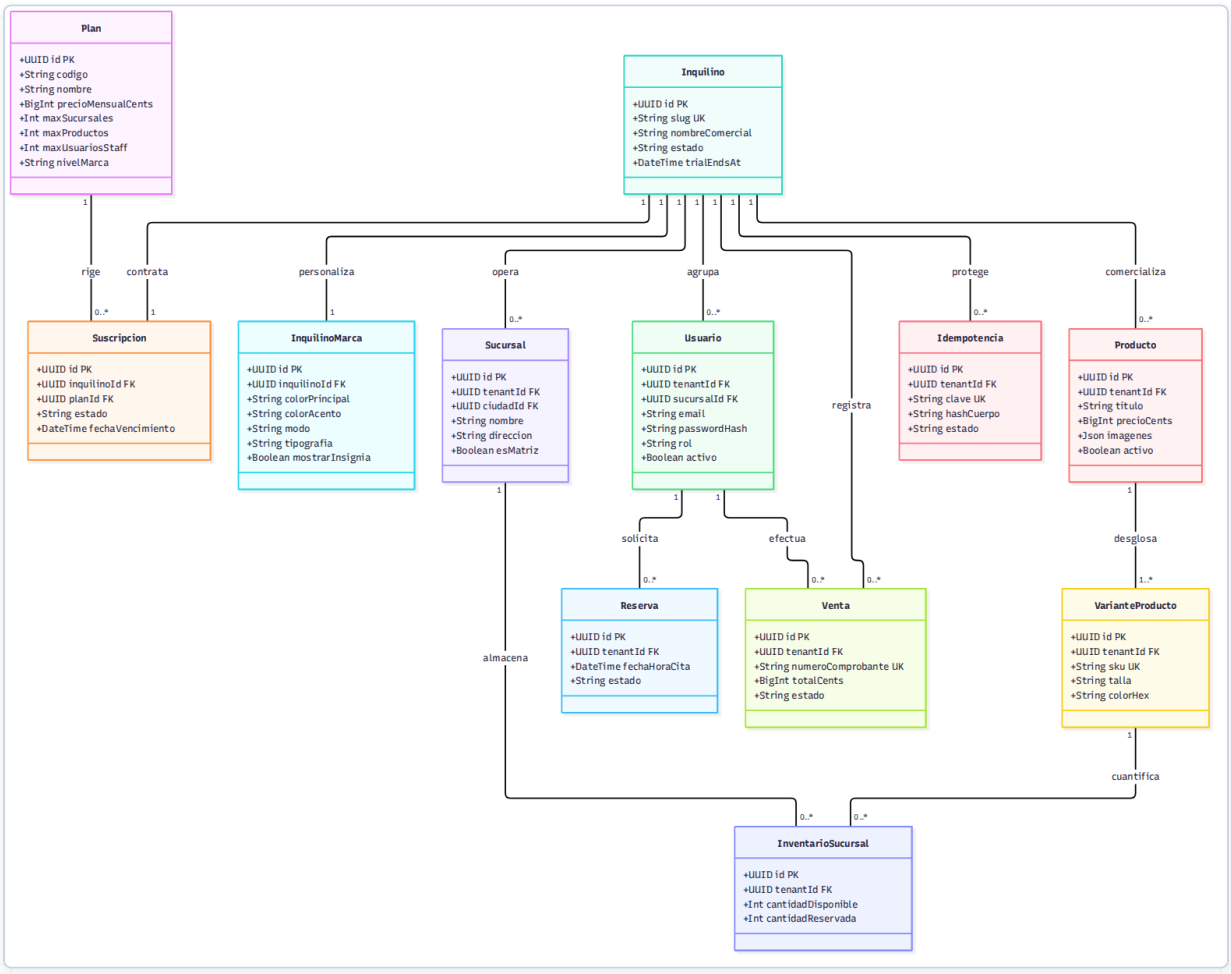


Figura 3.1: Modelo de Dominio de MirrorIA (UML 2.5)

3.2. Diagramas de Robustez (BCE)

Siguiendo el estándar de la metodología ICONIX, el **Análisis de Robustez** actúa como puente dinámico entre los casos de uso esenciales y el diseño detallado. Cada diagrama clasifica los elementos en tres estereotipos

formales: - `<<boundary>>`: Interfaces de usuario web (PWA), móviles nativas (Expo) o pasarelas externas. - `<<control>>`: Orquestadores de servicios de frontend (PWA Store/Hooks) y controladores de backend NestJS encargados de aplicar validaciones y reglas de negocio. - `<<entity>>`: Almacenes de persistencia local (IndexedDB / SQLite) y tablas de la base de datos central PostgreSQL.

CU01: Aprovisionar Tienda y Workspace SaaS

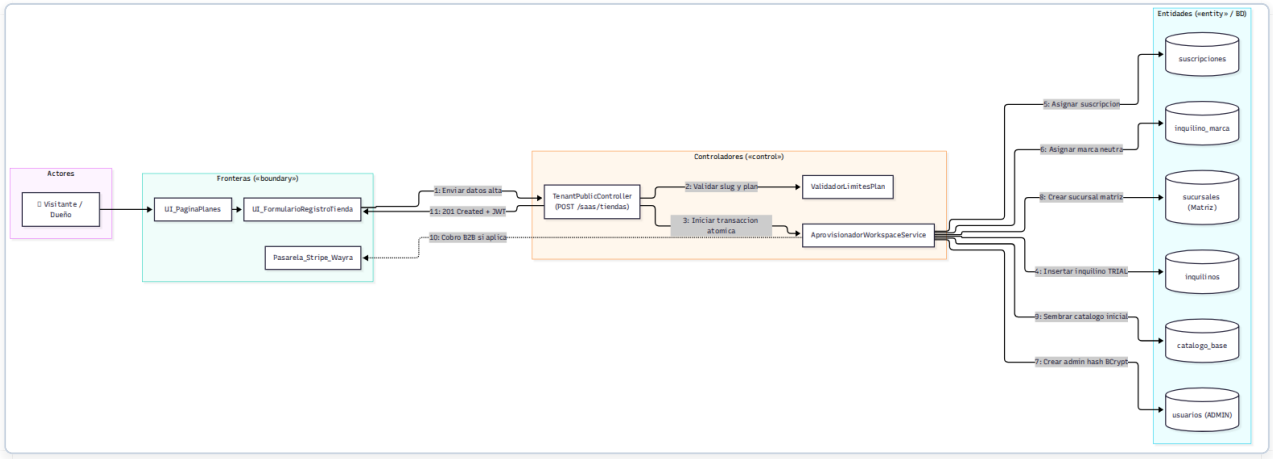


Figura 3.2: Diagrama de Robustez BCE — CU01 Aprovisionamiento SaaS

CU03: Personalizar Identidad Visual de Boutique (WCAG 2.1)

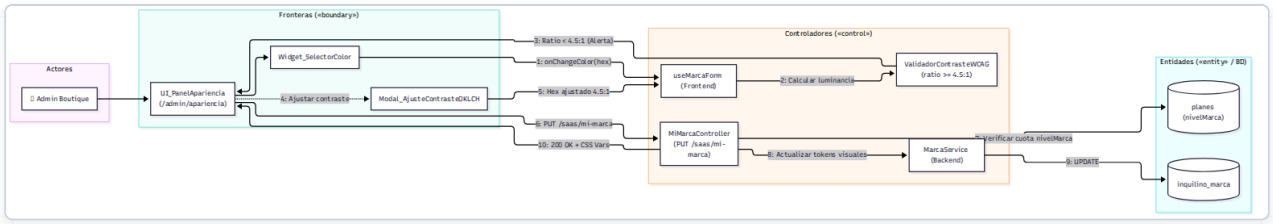


Figura 3.3: Diagrama de Robustez BCE — CU03 Identidad Visual WCAG 2.1

CU10: Operar en Modo Desconectado con Sincronización Segura

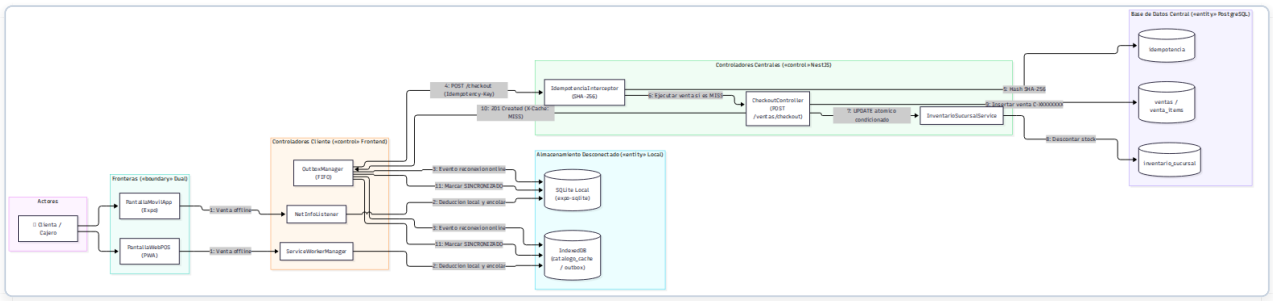


Figura 3.4: Diagrama de Robustez BCE Dual — CU10 Sincronización Offline e Idempotencia

---

CU11: Resolver Conflicto de Concurrencia de Stock (`stock = 1`)

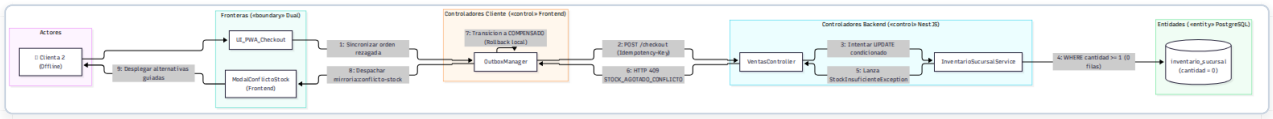


Figura 3.5: Diagrama de Robustez BCE Dual — CU11 Resolución Concurrente de Stock

---

3.3. Diagramas de Secuencia UML 2.5

Los diagramas de secuencia mantienen **isomorfismo estricto** con las líneas de vida identificadas en el análisis de robustez, modelando los mensajes HTTP, códigos de estado, parámetros de cabecera y bifurcaciones transaccionales.

Secuencia CU01: Aprovisionamiento Atómico SaaS

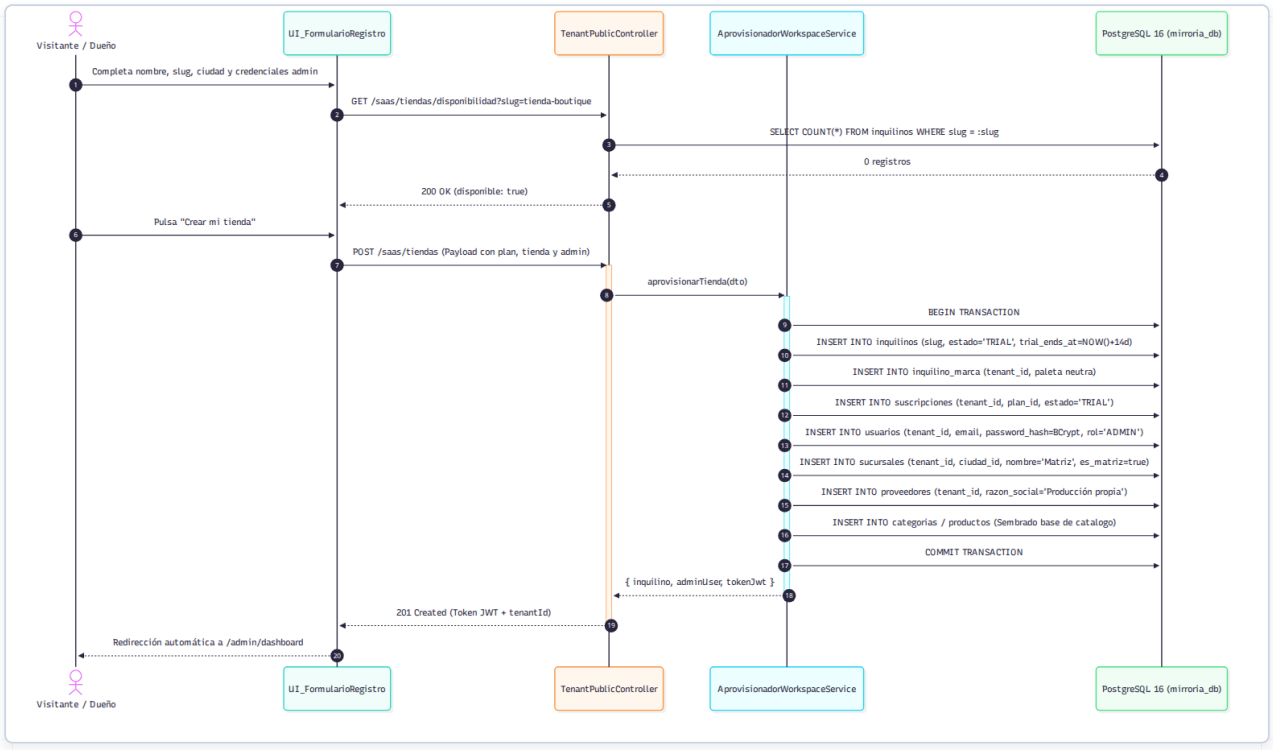


Figura 3.6: Diagrama de Secuencia UML — CU01 Aprovisionamiento de Tienda SaaS

Secuencia CU10: Operación Offline, Outbox e Idempotencia

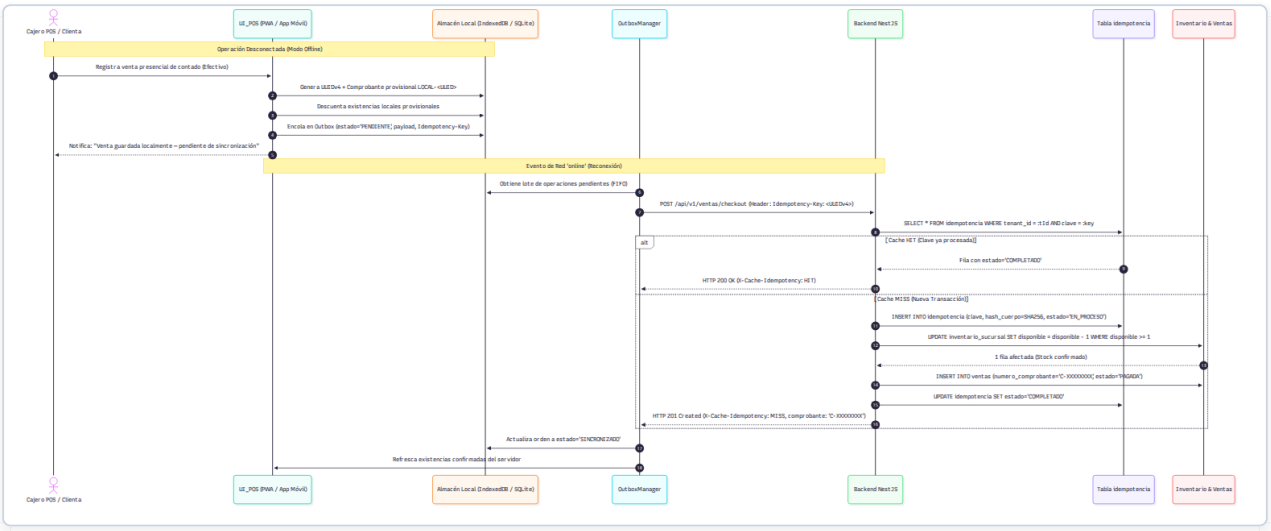


Figura 3.7: Diagrama de Secuencia UML — CU10 Operación Desconectada y Sincronización Outbox

---

Secuencia CU11: Conflicto Concurrente `stock = 1` y Compensación

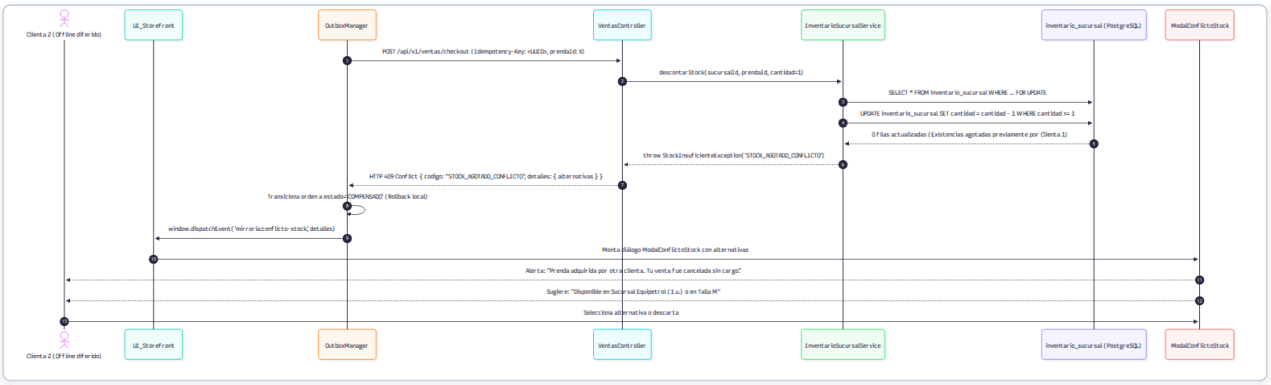


Figura 3.8: Diagrama de Secuencia UML — CU11 Conflicto Concurrente de Stock y Compensación

---

Secuencia CU16: Inferencia de Machine Learning con Fallback Heurístico

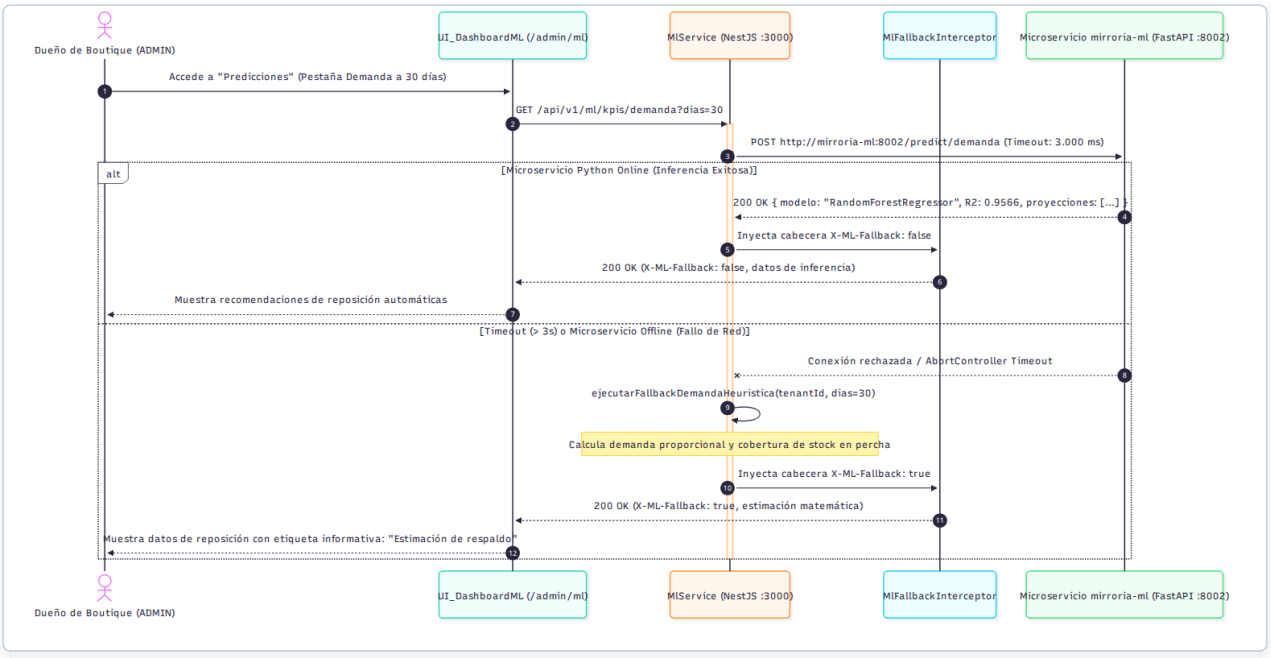


Figura 3.9: Diagrama de Secuencia UML — CU16 Inferencia Machine Learning con Fallback Heurístico

---

3.4. Diagrama de Clases de Análisis y Diseño UML

A continuación se modela la arquitectura de diseño estructurada en capas (`Boundary`, `Control` y `Entity`) para los casos de uso nucleares:

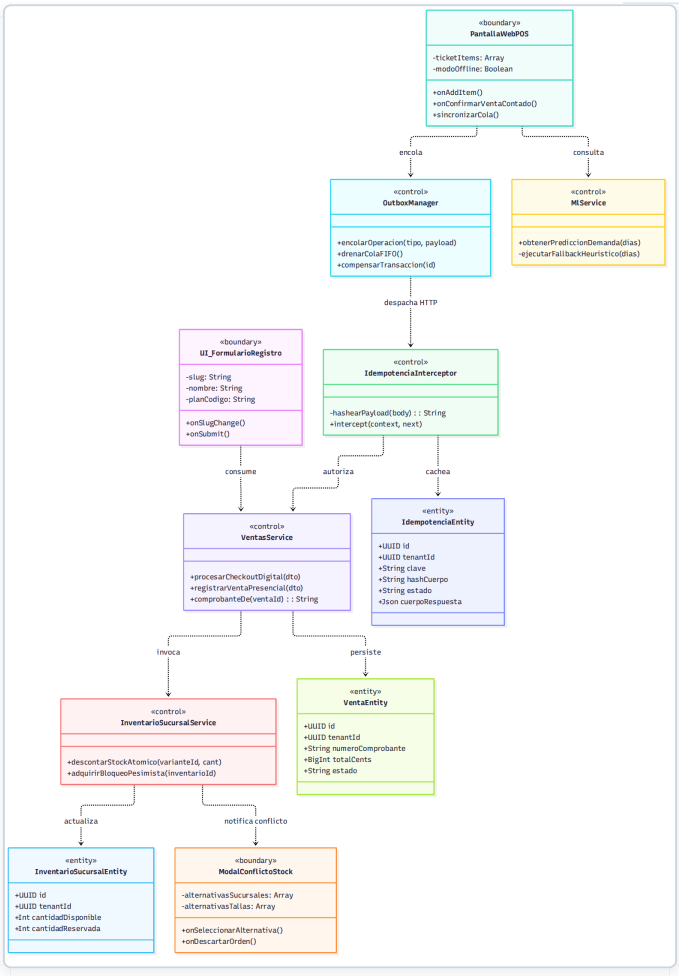


Figura 3.10: Diagrama de Clases de Análisis y Diseño UML 2.5

3.5. Diseño de la Base de Datos Relacional (PostgreSQL 16)

La base de datos relacional compartida (`mirroria_db`) se compone de **30 tablas normalizadas**, donde todas las entidades transaccionales y de negocio implementan `tenant_id: UUID NOT NULL` para aislamiento multi-tenant.

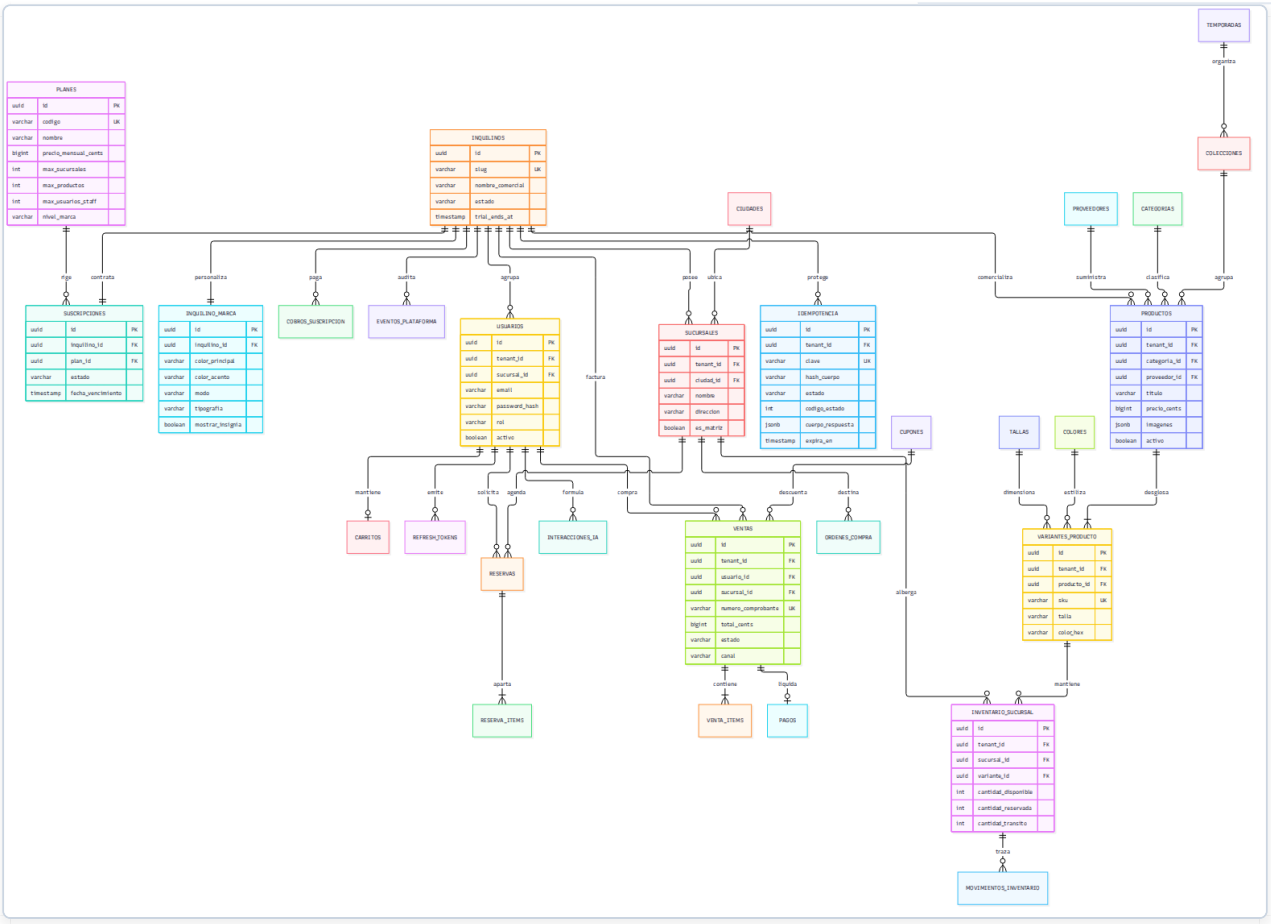


Figura 3.11: Diagrama Entidad-Relación Físico — 30 Tablas en PostgreSQL 16

3.6. Diseño de Interfaces de Usuario

Siguiendo el principio de **alta fidelidad y correspondencia con el código**, las interfaces se articulan en tres mundos funcionales claros:

1. **Consola Global de la Plataforma MirrorIA (`/plataforma/*`):**
2. Dashboard directivo con indicadores agregados de MRR, ARR, Churn Rate y volumen transaccionado (GMV).
3. Tabla administrativa de inquilinos con conmutador de estado (Activa / Suspendida), selector de plan comercial y botón de restablecimiento de marca de oficio.
4. **Tienda Web Pública y Vitrina Omnicanal (`/t/:slug/*`):**
5. Inyección dinámica de tokens de diseño en `:root` (logotipo, tipografías, radio de curvatura en botones y colores).
6. Ficha técnica interactiva de prendas con selector de variantes SKU, consulta de existencias por sucursal física e integración con el **Probador Virtual por IA** (Decart `lucy-vton-3.5`).
7. Carrito de compras persistente (`jsonb`) con tope de 3 unidades por variante y flujo de checkout con validación de cupones y pasarelas de pago (Stripe Checkout y Wayra QR dinámico).

8. **Panel Administrativo y Punto de Venta POS (`/admin/*`):**
9. **Personalización de Marca (`/admin/apariencia`):** Paleta interactiva con indicador visual accesible **WCAG 2.1** (alerta ámbar si el ratio es $< 4.5:1$ y botón de ajuste automático en espacio OKLCH).
10. **Consola Ejecutiva de BI (`/admin/dashboard` para ADMIN):** Resumen de ingresos netos con deltas simétricos, ticket promedio, rotación de inventario, tasa de conversión y gráficos de tendencia Recharts.
11. **Consola Operativa POS (`/admin/dashboard` para Staff):** Corte diario local en `America/La_Paz`, alertas visuales de existencias críticas (≤ 2 unidades) o agotadas, y agenda de citas de probador con recepción en un clic.
12. **Operativa Offline PWA:** Banner persistente `⚡ Sin conexión — operando con datos locales`, terminal POS con cobro de contado en efectivo y diálogo interactivo `ModalConflictoStock` con sugerencia guiada de alternativas.
13. **Consola de Predicciones y Machine Learning (`/admin/ml`):** Tarjetas ejecutivas en lenguaje de negocio para reposición a 30 días, semáforo de riesgo de inasistencia (*no-show*), segmentación RFM y panel lateral `FichaTecnicaSheet` con el rigor científico formal ($N=10.000$, `seed=42`).
14. **Reportes con IA y Excel (`/admin/reportes`):** Asistente conversacional en lenguaje natural, tabla dinámica interactiva con repivoteo en vivo y botón de descarga directa de balances contables a formato nativo `.xlsx` (`exceljs`).

CAPÍTULO IV. DESARROLLO E IMPLEMENTACIÓN

4.1. Herramientas y Tecnologías Utilizadas

Para la evolución, modelado e implementación de la plataforma SaaS multi-tenant **MirrorIA**, se seleccionó un conjunto de tecnologías y herramientas alineadas con los principios de la metodología ICONIX, priorizando la separación en capas, el alto rendimiento asíncrono, el aislamiento lógico riguroso y la tolerancia a fallos mediante persistencia distribuida:

4.1.1. Tecnologías de Desarrollo de Software

- **Backend Central: NestJS 12 y TypeScript (ESM nativo)**
- **TypeScript 6 & Node.js 24:** Lenguaje tipado que garantiza seguridad en tiempo de compilación y mantenibilidad corporativa. Se configuró en modo ECMAScript Modules puro (`"type": "module"`, `moduleResolution: "nodenext"`).
- **NestJS 12:** Framework empresarial basado en arquitectura modular e inyección de dependencias. Permite la estructuración desacoplada en 14 módulos de negocio, aplicación de filtros de excepción homogéneos (`GlobalExceptionFilter`), validación declarativa de esquemas mediante `class-validator` / `ValidationPipe` y documentación interactiva automática con Swagger OpenAPI (`/api/docs`).
- **TypeORM 1.x:** Mapeador objeto-relacional (ORM) encargado de la comunicación con PostgreSQL, aplicando la cláusula transversal `where: soloTenant({...})` y gestionando transacciones atómicas con bloqueos pesimistas (`pessimistic_write`).
- **Frontend Web y Punto de Venta (POS): React 19 (PWA)**
- **React 19 & Vite:** Biblioteca declarativa de interfaces de usuario optimizada con compilador nativo y Vite como empaquetador ultrarrápido (puerto local `5174` en desarrollo, Nginx en puerto `80` para producción).
- **Tailwind CSS v4 & shadcn/ui (Base UI):** Motor de diseño atómico sin dependencias obsoletas, utilizando el preset Nova sobre primitivas accesibles de Base UI y una iconografía uniforme al 100% en **Phosphor Icons** (`@phosphor-icons/react`).
- **IndexedDB (`idb`):** Motor de base de datos no relacional transaccional embebido en el navegador, utilizado por la PWA para almacenar localmente el catálogo de prendas (almacén `catalogo_cache` con TTL de 7 días) y la cola de operaciones diferidas (almacén `outbox` FIFO).
- **Service Worker (`sw.js`):** Script en segundo plano que gestiona la estrategia de caché *Cache-First* para el App Shell (`mirroria-app-shell-v1`) y *Network-First* con fallback a IndexedDB para el consumo de la API REST.
- **Persistencia Central: PostgreSQL 16 (`mirroria_db`)**
- Motor relacional transaccional robusto ejecutándose en contenedor Docker (`mirroria_postgres`, puerto `5435`). Alberga las 30 tablas del sistema con extensiones criptográficas `pgcrypto` (`gen_random_uuid()`) y `uuid-ossp`. Garantiza el aislamiento multi-tenant mediante índices compuestos sobre `tenant_id` y transacciones con aislamiento ACID.

- **Microservicio de Machine Learning: Python 3.12 y FastAPI**
- **FastAPI:** Microframework web asíncrono en Python que expone endpoints REST en puerto `8002` (contenedor `mirroria_ml`, 68 MB de RAM) para inferencia analítica en tiempo real.
- **Scikit-Learn, Pandas y Joblib:** Bibliotecas estadísticas empleadas para entrenar de forma reproducible ($N=10.000$, semilla fija `seed=42`) los cuatro modelos: Random Forest (M1), Regresión Logística L2 (M2), K-Means (M3) e Isolation Forest (M4).
- **Movilidad y Persistencia Embebida: React Native / Expo SDK 57**
- Cliente móvil nativo con **SQLite** (`expo-sqlite`) para el catálogo, el carrito y la cola de compras, y `@react-native-community/netinfo` para detectar la reconexión y sincronizar.
- **Servicios e Integraciones Externas:**
- **PayPal (Orders v2, REST):** Pasarela internacional en sandbox. Aprobar la orden no cobra: el servidor la captura al volver la clienta (`POST /pagos/ventas/:id/confirmar-tarjeta`, `/saas/mi-suscripcion/cobro/:id/confirmar`) y acepta avisos firmados en `/saas/webhooks/paypal`.
- **Stripe Checkout:** Pasarela de pagos internacionales para tarjetas de crédito/débito con verificación de webhooks firmados (`Stripe-Signature`) y confirmación con Stripe al regreso; cobra en USD convirtiendo con `TIPO_CAMBIO_BOB_USD`.
- **Banco Simulado Wayra (QR):** Banco simulado (alternativa a la API de un banco real, no accesible para el proyecto) que emite códigos QR dinámicos y callbacks firmados mediante HMAC-SHA256 con idempotencia por `event_id`.
- **Decart AI (`lucy-vton-3.5`):** Motor generativo en la nube para el probador virtual en tiempo real consumido a través de tokens efímeros seguros de 5 minutos.
- **DeepSeek LLM & DuckDB:** Modelo de lenguaje para extracción analítica de lenguaje natural y motor in-process DuckDB (`:memory:`) para simulaciones *What-If*.
- **ExcelJS:** Motor de generación de libros de cálculo nativos en formato `.xlsx` con formato contable institucional `"Bs" #,##0.00`.

4.1.2. Herramientas de Modelado, Diseño y Productividad

- **Mermaid & Mermaid CLI (`@mermaid-js/mermaid-cli` v12.2.0):** Herramienta de diagramación declarativa como código utilizada para modelar los diagramas de clases, robustez BCE, secuencias UML 2.5 y diagramas C4, compilándolos de forma nativa a imágenes PNG de alta resolución.
- **Docker & Docker Compose:** Entorno de contenedores para orquestar la suite de servicios (`mirroria_postgres`, `mirroria_backend`, `mirroria_frontend` y `mirroria_ml`) con políticas `restart: "no"` para cuidar la memoria RAM en WSL3.
- **Vitest & Oxlint:** Suite de pruebas ultrarrápida (557 pruebas en backend, 345 en frontend y 17 en móvil) y linter de alto rendimiento.
- **Git & GitHub:** Control de versiones distribuido bajo estrategia de rama única `main` para trazabilidad de código.

4.2. Diseño de la Arquitectura del Sistema (Diagrama C4 Nivel 1: Contexto)

El siguiente diagrama de Contexto (C4 Nivel 1) modela las fronteras del sistema MirrorIA, sus usuarios y las interacciones con plataformas externas:

4.3. Implementación de la Base de Datos Relacional (PostgreSQL 16)

A continuación se expone el script DDL SQL representativo de la base de datos compartida `mirroria_db`, reflejando la extensión `pgcrypto`, llaves primarias basadas en UUID, discriminador multi-tenant `tenant_id` e índices de integridad:

```
-- Extensión criptográfica para UUIDv4 nativo
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- 1. TABLA: planes (Catálogo de suscripciones comerciales SaaS)
CREATE TABLE public.planes (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    codigo character varying(50) NOT NULL UNIQUE,
    nombre character varying(100) NOT NULL,
    precio_mensual_cents bigint NOT NULL,
    precio_mensual_usd_cents bigint NOT NULL,
    max_sucursales integer,
    max_productos integer,
    max_usuarios_staff integer,
    dias_prueba integer DEFAULT 14 NOT NULL,
    destacado boolean DEFAULT false NOT NULL,
    nivel_marca character varying(20) DEFAULT 'ESENCIAL' NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_planes PRIMARY KEY (id)
);

-- 2. TABLA: inquilinos (Empresas boutiques contratantes)
CREATE TABLE public.inquilinos (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    slug character varying(80) NOT NULL UNIQUE,
    nombre_comercial character varying(150) NOT NULL,
    estado character varying(20) DEFAULT 'TRIAL' NOT NULL,
    trial_ends_at timestamp without time zone NOT NULL,
    plan_id uuid NOT NULL REFERENCES public.planes(id),
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_inquilinos PRIMARY KEY (id)
);

-- 3. TABLA: suscripciones (Contratos B2B de inquilinos con planes)
CREATE TABLE public.suscripciones (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL UNIQUE REFERENCES public.inquilinos(id) ON DELETE CASCADE,
    plan_id uuid NOT NULL REFERENCES public.planes(id),
    estado character varying(20) DEFAULT 'TRIAL' NOT NULL,
    fecha_inicio timestamp without time zone DEFAULT now() NOT NULL,
    fecha_vencimiento timestamp without time zone NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_suscripciones PRIMARY KEY (id)
);

-- 4. TABLA: inquilino_marca (Tokens de personalización visual con WCAG)
CREATE TABLE public.inquilino_marca (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL UNIQUE REFERENCES public.inquilinos(id) ON DELETE CASCADE,
    logotipo_url character varying(255),
    favicon_url character varying(255),
    color_principal character varying(7) DEFAULT '#281B89' NOT NULL,
    color_acento character varying(7) DEFAULT '#C3F74C' NOT NULL,
```

```
    modo character varying(10) DEFAULT 'CLARO' NOT NULL,
    tipografia character varying(30) DEFAULT 'outfit' NOT NULL,
    radio_borde character varying(10) DEFAULT 'md' NOT NULL,
    mostrar_insignia boolean DEFAULT true NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_inquilino_marca PRIMARY KEY (id)
);

-- 5. TABLA: ciudades (Catálogo global de capitales de Bolivia)
CREATE TABLE public.ciudades (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    nombre character varying(100) NOT NULL UNIQUE,
    codigo_iso character varying(10) NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_ciudades PRIMARY KEY (id)
);

-- 6. TABLA: sucursales (Puntos de venta físicos por boutique)
CREATE TABLE public.sucursales (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    ciudad_id uuid NOT NULL REFERENCES public.ciudades(id),
    nombre character varying(150) NOT NULL,
    direccion character varying(255) NOT NULL,
    es_matriz boolean DEFAULT false NOT NULL,
    activa boolean DEFAULT true NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_sucursales PRIMARY KEY (id)
);
CREATE INDEX idx_sucursales_tenant ON public.sucursales(tenant_id);

-- 7. TABLA: usuarios (Cuentas con aislamiento multitenant y RBAC)
CREATE TABLE public.usuarios (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid, -- NULL exclusivamente para DUENO_PLATAFORMA
    sucursal_id uuid REFERENCES public.sucursales(id),
    email character varying(150) NOT NULL,
    password_hash character varying(255) NOT NULL,
    nombre character varying(100) NOT NULL,
    rol character varying(30) NOT NULL,
    activo boolean DEFAULT true NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_usuarios PRIMARY KEY (id),
    CONSTRAINT uk_usuarios_tenant_email UNIQUE (tenant_id, email)
);
CREATE INDEX idx_usuarios_tenant ON public.usuarios(tenant_id);

-- 8. TABLA: proveedores (Talleres de confección textil)
CREATE TABLE public.proveedores (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    razon_social character varying(150) NOT NULL,
```

```
    nit character varying(30),
    contacto character varying(100),
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_proveedores PRIMARY KEY (id)
);
CREATE INDEX idx_proveedores_tenant ON public.proveedores(tenant_id);

-- 9. TABLA: categorias (Taxonomía de indumentaria femenina)
CREATE TABLE public.categorias (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    nombre character varying(100) NOT NULL,
    slug character varying(120) NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_categorias PRIMARY KEY (id),
    CONSTRAINT uk_categorias_tenant_slug UNIQUE (tenant_id, slug)
);
CREATE INDEX idx_categorias_tenant ON public.categorias(tenant_id);

-- 10. TABLA: productos (Prendas maestras)
CREATE TABLE public.productos (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    categoria_id uuid NOT NULL REFERENCES public.categorias(id),
    proveedor_id uuid REFERENCES public.proveedores(id),
    titulo character varying(200) NOT NULL,
    descripcion text,
    precio_cents bigint NOT NULL,
    imagenes jsonb DEFAULT '[]'::jsonb NOT NULL,
    activo boolean DEFAULT true NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_productos PRIMARY KEY (id)
);
CREATE INDEX idx_productos_tenant ON public.productos(tenant_id);

-- 11. TABLA: variantes_producto (Matriz SKU por talla y color)
CREATE TABLE public.variantes_producto (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    producto_id uuid NOT NULL REFERENCES public.productos(id) ON DELETE CASCADE,
    sku character varying(100) NOT NULL,
    talla character varying(20) NOT NULL,
    color_hex character varying(7) NOT NULL,
    color_nombre character varying(50) NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_variantes PRIMARY KEY (id),
    CONSTRAINT uk_variantes_tenant_sku UNIQUE (tenant_id, sku)
);
CREATE INDEX idx_variantes_tenant ON public.variantes_producto(tenant_id);

-- 12. TABLA: inventario_sucursal (Control de existencias físicas)
CREATE TABLE public.inventario_sucursal (
```

```
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    sucursal_id uuid NOT NULL REFERENCES public.sucursales(id),
    variante_id uuid NOT NULL REFERENCES public.variantes_producto(id),
    cantidad_disponible integer DEFAULT 0 NOT NULL,
    cantidad_reservada integer DEFAULT 0 NOT NULL,
    cantidad_transito integer DEFAULT 0 NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_inventario PRIMARY KEY (id),
    CONSTRAINT uk_inventario_sucursal_variante UNIQUE (tenant_id, sucursal_id, variante_id),
    CONSTRAINT chk_disponible_no_negativo CHECK (cantidad_disponible >= 0),
    CONSTRAINT chk_reservada_no_negativo CHECK (cantidad_reservada >= 0)
);
CREATE INDEX idx_inventario_tenant ON public.inventario_sucursal(tenant_id);

-- 13. TABLA: movimientos_inventario (Kardex inmutable)
CREATE TABLE public.movimientos_inventario (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    inventario_id uuid NOT NULL REFERENCES public.inventario_sucursal(id),
    tipo character varying(30) NOT NULL, -- VENTA, RESERVA, RECEPCION, AJUSTE
    cantidad integer NOT NULL,
    saldo_anterior integer NOT NULL,
    saldo_nuevo integer NOT NULL,
    motivo character varying(255),
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_movimientos PRIMARY KEY (id)
);
CREATE INDEX idx_movimientos_tenant ON public.movimientos_inventario(tenant_id);

-- 14. TABLA: ordenes_compra (Abastecimiento textil a talleres)
CREATE TABLE public.ordenes_compra (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    proveedor_id uuid NOT NULL REFERENCES public.proveedores(id),
    sucursal_destino_id uuid NOT NULL REFERENCES public.sucursales(id),
    usuario_id uuid NOT NULL REFERENCES public.usuarios(id),
    estado character varying(30) DEFAULT 'PENDIENTE' NOT NULL,
    items jsonb DEFAULT '[]'::jsonb NOT NULL,
    fecha_pedido timestamp without time zone DEFAULT now() NOT NULL,
    fecha_recepcion timestamp without time zone,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_ordenes_compra PRIMARY KEY (id)
);
CREATE INDEX idx_ordenes_tenant ON public.ordenes_compra(tenant_id);

-- 15. TABLA: carritos (Bolsa de compras digital persistente)
CREATE TABLE public.carritos (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    usuario_id uuid NOT NULL UNIQUE REFERENCES public.usuarios(id) ON DELETE CASCADE,
    items jsonb DEFAULT '[]'::jsonb NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
```

```
    CONSTRAINT pk_carritos PRIMARY KEY (id)
);
CREATE INDEX idx_carritos_tenant ON public.carritos(tenant_id);

-- 16. TABLA: ventas (Órdenes omnicanal POS y Web)
CREATE TABLE public.ventas (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    sucursal_id uuid NOT NULL REFERENCES public.sucursales(id),
    cliente_id uuid REFERENCES public.usuarios(id),
    cajero_id uuid REFERENCES public.usuarios(id),
    reserva_id uuid,
    cupon_id uuid,
    canal character varying(20) NOT NULL, -- WEB o POS
    estado character varying(30) DEFAULT 'PENDIENTE' NOT NULL, -- PENDIENTE, PAGADA, CANCELA-
DA
    numero_comprobante character varying(50),
    subtotal_cents bigint NOT NULL,
    descuento_cents bigint DEFAULT 0 NOT NULL,
    total_cents bigint NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_ventas PRIMARY KEY (id)
);
CREATE INDEX idx_ventas_tenant ON public.ventas(tenant_id);

-- 17. TABLA: venta_items (Líneas de detalle de venta)
CREATE TABLE public.venta_items (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    venta_id uuid NOT NULL REFERENCES public.ventas(id) ON DELETE CASCADE,
    variante_id uuid NOT NULL REFERENCES public.variantes_producto(id),
    cantidad integer NOT NULL,
    precio_unitario_cents bigint NOT NULL,
    subtotal_cents bigint NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_venta_items PRIMARY KEY (id)
);
CREATE INDEX idx_venta_items_tenant ON public.venta_items(tenant_id);

-- 18. TABLA: reservas (Citas de prueba física en tienda)
CREATE TABLE public.reservas (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    sucursal_id uuid NOT NULL REFERENCES public.sucursales(id),
    cliente_id uuid NOT NULL REFERENCES public.usuarios(id),
    fecha_hora_cita timestamp without time zone NOT NULL,
    estado character varying(30) DEFAULT 'PENDIENTE' NOT NULL, -- PENDIENTE, CONFIRMADA,
EN_TIENDA, COMPLETADA, EXPIRADA, NO_SHOW, CANCELADA
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_reservas PRIMARY KEY (id)
);
CREATE INDEX idx_reservas_tenant ON public.reservas(tenant_id);

-- 19. TABLA: reserva_items (Prendas retenidas en probador)
```

```
CREATE TABLE public.reserva_items (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    reserva_id uuid NOT NULL REFERENCES public.reservas(id) ON DELETE CASCADE,
    variante_id uuid NOT NULL REFERENCES public.variantes_producto(id),
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_reserva_items PRIMARY KEY (id)
);
CREATE INDEX idx_reserva_items_tenant ON public.reserva_items(tenant_id);

-- 20. TABLA: pagos (Registro de cobros con pasarelas)
CREATE TABLE public.pagos (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    venta_id uuid NOT NULL REFERENCES public.ventas(id),
    proveedor_pago character varying(30) NOT NULL, -- STRIPE, WAYRA, MANUAL
    metodo character varying(30) NOT NULL, -- TARJETA, QR, EFECTIVO
    monto_cents bigint NOT NULL,
    estado character varying(30) DEFAULT 'PENDIENTE' NOT NULL, -- PENDIENTE, APROBADO, RECHA-
ZADO, REEMBOLSADO
    monto_reembolsado_cents bigint DEFAULT 0 NOT NULL,
    event_id character varying(150) NOT NULL UNIQUE,
    referencia_externa character varying(150),
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_pagos PRIMARY KEY (id)
);
CREATE INDEX idx_pagos_tenant ON public.pagos(tenant_id);

-- 21. TABLA: idempotencia (Garantía transaccional offline)
CREATE TABLE public.idempotencia (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    clave character varying(150) NOT NULL,
    hash_cuerpo character varying(64) NOT NULL,
    ruta character varying(255) NOT NULL,
    metodo character varying(10) NOT NULL,
    estado character varying(20) DEFAULT 'EN_PROCESO' NOT NULL, -- EN_PROCESO, COMPLETADO,
FALLIDO
    codigo_estado integer,
    cuerpo_respuesta jsonb,
    expira_en timestamp without time zone,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_idempotencia PRIMARY KEY (id),
    CONSTRAINT uk_idempotencia_tenant_clave UNIQUE (tenant_id, clave)
);
CREATE INDEX idx_idempotencia_tenant ON public.idempotencia(tenant_id);

-- 22. TABLA: cobros_suscripcion (Cobranzas B2B de planes SaaS)
CREATE TABLE public.cobros_suscripcion (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    plan_id uuid NOT NULL REFERENCES public.planes(id),
    proveedor_pago character varying(30) NOT NULL,
    monto_cents bigint NOT NULL,
```

```
    estado character varying(30) DEFAULT 'PENDIENTE' NOT NULL,
    event_id character varying(150) NOT NULL UNIQUE,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_cobros_suscripcion PRIMARY KEY (id)
);

-- 23. TABLA: cupones (Descuentos y fidelización)
CREATE TABLE public.cupones (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    codigo character varying(50) NOT NULL,
    tipo character varying(20) NOT NULL, -- PORCENTAJE o MONTO_FIJO
    valor integer NOT NULL,
    monto_minimo_cents bigint DEFAULT 0 NOT NULL,
    usos_maximos integer DEFAULT 100 NOT NULL,
    usos_actuales integer DEFAULT 0 NOT NULL,
    valido_desde timestamp without time zone NOT NULL,
    valido_hasta timestamp without time zone NOT NULL,
    activo boolean DEFAULT true NOT NULL,
    "createdAt" timestamp without time zone DEFAULT now() NOT NULL,
    "updatedAt" timestamp without time zone DEFAULT now() NOT NULL,
    CONSTRAINT pk_cupones PRIMARY KEY (id),
    CONSTRAINT uk_cupones_tenant_codigo UNIQUE (tenant_id, codigo)
);

-- 24 a 30: Tablas de soporte (temporadas, colecciones, tallas, colores, refresh_tokens, in-
teracciones_ia, eventos_plataforma)
-- Mantienen id UUID, createdAt, updatedAt y tenant_id según corresponda.
```

4.4. Desarrollo de los Módulos del Sistema

La implementación del sistema se organiza de forma estricta en torno a los **cinco módulos funcionales rectores**:

4.4.1. Módulo de Gestión Multitenant y Suscripciones (SaaS)

- **Lógica Backend:** Implementado en `SaasModule`. Administra las tablas `planes`, `inquilinos`, `suscripciones`, `inquilino_marca` y `cobros_suscripcion`. El servicio `AprovisionadorWorkspaceService` orquesta la transacción atómica de alta de boutique sembrando la base relacional en PostgreSQL. El método `assertDentroDelPlan` valida en tiempo de ejecución las cuotas máximas de sucursales, productos y staff.
- **Lógica Frontend:** Vistas `/plataforma/*` para el superusuario y `/admin/suscripcion` para el cliente del SaaS, con la vista `/admin/apariencia` que implementa la validación algorítmica de contraste accesible WCAG 2.1 en espacio cromático OKLCH.

4.4.2. Módulo de Catálogo, Inventario y Puntos de Venta (POS)

- **Lógica Backend:** Implementado mediante los módulos desacoplados `catalogo`, `inventario`, `proveedores` y `ventas`. Administra existencias físicas mediante el servicio `InventarioSucursalService`,

asegurando que `ajustarStock` sea el único punto que toque `cantidad_disponible` con sentencias SQL `UPDATE` condicionadas.

- **Lógica Frontend:** Interfaz de punto de venta (POS) en `/admin/dashboard` y `/admin/cobros` para el Cajero, con lectura rápida de existencias, confirmación de cobros en efectivo y emisión determinista de comprobantes `C-XXXXXXXX`.

4.4.3. Módulo de Experiencia del Cliente y Probador Virtual por IA

- **Lógica Backend:** Módulos `probador`, `reservas` y `promociones`. El módulo `probador` actúa como gateway hacia el motor generativo Decart (`lucy-vton-3.5`), expidiendo tokens de acceso efímeros de 5 minutos con limitador de tasa por IP. El servicio `ReservasService` implementa la máquina de estados finitos (`PENDIENTE` $\to$ `CONFIRMADA` $\to$ `EN_TIENDA` $\to$ `COMPLETADA`) y la suspensión de 30 días ante 2 faltas consecutivas.
- **Lógica Frontend:** Vitrina pública `/t/:slug` con inyección dinámica de variables CSS de marca, ficha técnica de prenda con selector SKU e interfaz interactiva del Probador Virtual VTON en `/t/:slug/probador`.

4.4.4. Módulo de Operación Offline-First y Sincronización

- **Lógica Cliente:** El archivo `public/sw.js` cachea el App Shell en `mirroria-app-shell-v1`. El módulo `src/lib/offline/idb.ts` estructura los almacenes IndexedDB `catalogo_cache` (TTL 7 días) y `outbox` FIFO. La clase `OutboxManager` detecta eventos de reconexión y drena la cola enviando la cabecera `Idempotency-Key` (SHA-256). Ante rechazos 409 por stock agotado, transiciona a `COMPENSADO` y abre `ModalConflictoStock`.
- **Lógica Servidor:** El interceptor `IdempotenciaInterceptor` extrae la llave UUIDv4 y el hash canónico del body, consultando la tabla `idempotencia`. Si la orden ya fue procesada, responde con `X-Cache-Idempotency: HIT`.

4.4.5. Módulo de Reportes Generativos con IA y Machine Learning

- **Lógica Backend:** Módulo `ia` con DeepSeek LLM y catálogo de 23 métricas parametrizadas (sin sentencias SQL arbitrarias generadas por IA) y motor in-process DuckDB (`:memory:`) para simulaciones *What-If*. Módulo `bi` con endpoints analíticos directos `/dashboard/ejecutivo` y `/dashboard/operativo`. Módulo `ml` que actúa como proxy hacia el microservicio Python en puerto `8002` con fallback heurístico resiliente (`X-ML-Fallback: true`).
- **Lógica Microservicio ML (`mirroria-ml`):** Proceso en FastAPI con 4 modelos entrenados reproducibles ($N=10.000$, `seed=42`): Demanda a 30 días (Random Forest, $R^2 = 0.9566$), Riesgo No-Show (Regresión Logística L2, ROC-AUC 0.7978), Segmentación RFM (K-Means, Silueta 0.6823) y Auditoría POS (Isolation Forest, c=0.03).
- **Lógica Frontend:** Tableros `/admin/dashboard` y `/admin/ml` en lenguaje directivo con gráficos Recharts y modal lateral de rigor científico formal `FichaTecnicaSheet`.

4.5. Integración de Componentes

La comunicación y cohesión entre los diferentes subsistemas se rige por contratos estrictos: 1. **Frontend $\leftrightarrow$ Backend Central (NestJS):** Comunicación REST JSON sobre HTTP/S en `/api/v1/*`. Las peticiones inyectan obligatoriamente el contexto mediante la cabecera `X-Tenant: <slug>` o el token Bearer JWT con el `tenantId` verificado contra la base de datos. 2. **Backend NestJS $\leftrightarrow$ Microservicio Python ML:** Llamadas HTTP internas síncronas hacia `http://mirroria-ml:8002/predict/*` con un tiempo de espera límite estricto (*timeout*) de **1.500 ms** gestionado mediante `AbortController`. Ante caídas o sobrecargas, el interceptor conmuta de forma transparente al cálculo heurístico matemático de respaldo. 3. **Backend NestJS $\leftrightarrow$ Pasarelas de Pago:** - **Stripe:** Creación de sesiones alojadas mediante el SDK oficial y recepción de webhooks en `POST /api/v1/pagos/webhook` con validación obligatoria de la firma criptográfica `Stripe-Signature`. - **Banco Wayra:** Generación de códigos QR con payload estructurado y recepción de avisos bancarios en `POST /api/v1/pagos/qr/aviso` con verificación estricta de firma **HMAC-SHA256** mediante `crypto.timingSafeEqual`. 4. **Cliente Web $\leftrightarrow$ Persistencia Local y Outbox:** Sincronización asíncrona no bloqueante coordinada entre el hilo principal de React 19 y el Service Worker mediante eventos globales de ventana (`mirroria:conflicto-stock`).

CAPÍTULO V. PRUEBAS Y RESULTADOS

5.1. Plan de Pruebas

El objetivo primordial del plan de pruebas es garantizar que la plataforma **MirrorIA** cumpla con los requisitos funcionales (RF01 a RF26), los requisitos no funcionales (RNF01 a RNF12) y las 14 reglas de negocio bajo condiciones de estrés, concurrencia y fallas de conectividad.

El alcance de las pruebas cubre los cinco módulos del sistema: 1. **Gestión Multitenant y Suscripciones:** Aislamiento de datos con `tenant_id`, control de cuotas por plan comercial y cobros recurrentes B2B. 2. **Catálogo, Inventario y Puntos de Venta (POS):** Bloqueos pesimistas en deducción de existencias y emisión de comprobantes `C-XXXXXXXX`. 3. **Experiencia del Cliente y Probador Virtual IA:** Generación segura de tokens efímeros para Decart AI y flujo de citas para probador físico. 4. **Operación Offline-First y Sincronización:** Persistencia en IndexedDB / SQLite, protocolo de idempotencia SHA-256 y resolución determinista del conflicto concurrente `stock = 1`. 5. **Reportes con IA y Machine Learning:** Precisión de inferencia en Python FastAPI (:8002) y fallback heurístico resiliente ante caídas.

Tabla 5.1: Modelo de Plan de Pruebas por Módulo

Módulo	Tipo de Prueba	Objetivo Principal	Herramienta / Entorno	Criterio de Aceptación	Responsable
Multitenant y Suscripciones	Funcional / Seguridad	Verificar que ninguna petición con JWT de una tienda acceda a registros de otro inquilino y certificar el control de cuotas del plan.	Vitest / Python (`qa_aislamiento.py`)	100% de peticiones cruzadas devuelven HTTP 403 o 404. Control de cuotas rechaza excedentes con HTTP 409.	Equipo Backend
Catálogo e Inventario	Integración / Concurrencia	Certificar que `UPDATE` condicionado previene inventarios negativos bajo concurrencia pesimista.	Vitest e2e / PostgreSQL 16	Cero existencias negativas en `cantidad_disponible` y registro inmutable en `movimientos_inventario`.	Equipo Backend
Operación Offline y Sync	Resiliencia / Integración	Validar persistencia en IndexedDB, drenado por outbox FIFO, idempotencia con hash SHA-256 y resolución del conflicto `stock = 1`.	Vitest / Playwright en PWA sin red	Reenvíos sucesivos retornan `X-Cache-Idempotency: HIT`. Colisión en última prenda emite HTTP 409 tipado y activa rollback compensatorio.	Equipo Frontend / QA
Experiencia y Probador IA	Seguridad / Funcional	Validar que la emisión de credenciales de Decart AI no exponga API keys y cumpla el límite de tasa por IP.	Vitest / Supertest	Token efímero expira a los 5 minutos; peticiones > 10 en 10 min reciben HTTP 429.	Equipo Backend
Machine Learning y Analítica	Rendimiento / Disponibilidad	Validar métricas de modelos (M1 a M4) y verificar activación automática del fallback heurístico ante caída del servicio en :8002.	Pytest / Vitest (`MlFallbackInterceptor`)	Timeout ≤ 1.500 ms; al apagar el contenedor se emite `X-ML-Fallback: true` con estimación matemática coherente.	Equipo ML / Backend

---

5.2. Pruebas Funcionales

Las pruebas funcionales verifican el comportamiento de cada Caso de Uso ante entradas válidas, flujos alternativos y violaciones de reglas de negocio:

Tabla 5.2: Registro de Pruebas Funcionales Ejecutadas

ID	Caso de Uso / RF	Descripción de la Prueba	Resultado Esperado	Resultado Obtenido	Estado
PF-01	CU01 / RF01	Registrar una nueva boutique mediante el endpoint público `/saas/tiendas` con slug único.	Aprovisionamiento atómico: crea inquilino, marca, sucursal, proveedor propio y usuario `ADMIN` en estado `TRIAL` (14 días).	Aprovisionamiento completado en 112 ms; emite token JWT y redirige a `/admin/dashboard`.	**Aprobado**
PF-02	CU02 / RF03	Crear una segunda sucursal en una boutique suscrita al Plan Básico (`maxSucursales = 1`).	El sistema rechaza la operación con código HTTP 409 informando que se alcanzó el límite del plan.	Emite HTTP 409 `LimitePlanExcedidoException` indicando el límite del plan; seis altas simultáneas tampoco lo superan (bloqueo por tienda).	**Aprobado**
PF-03	CU03 / RF11	Configurar un color principal de tienda con contraste deficiente contra el fondo blanco (ej. `#FFFF00`).	El sistema calcula el ratio WCAG 2.1 ($< 4.5:1$), emite alerta visual y bloquea el guardado.	Validador emite ratio 1.07:1, muestra alerta ámbar y herramienta OKLCH recalcula automáticamente a color accesible.	**Aprobado**
PF-04	CU07 / RF14	Intentar agregar 4 unidades de una misma variante SKU al carrito de compras digital.	La validación del DTO y del servicio rechaza la solicitud limitando a máximo 3 unidades por variante.	Rechaza con HTTP 409 (regla de negocio) indicando el tope de 3 unidades por talla y color.	**Aprobado**
PF-05	CU09 / RF15	Solicitar una cita de probador con 4 prendas o con fecha fuera del rango de 48 horas.	El sistema rechaza la creación de la reserva por violación de los límites anti-acaparamiento.	Rechaza con HTTP 409 (regla de negocio) por el tope de prendas y la ventana de 48 h.	**Aprobado**
PF-06	CU09 / RF15	Clienta con 2 inasistencias acumuladas (`NO_SHOW`) intenta agendar una nueva cita.	El sistema constata las 2 faltas en los últimos 30 días y bloquea la creación por 30 días.	Emite HTTP 403 `"Usuario bloqueado para reservas hasta YYYY-MM-DD"`.	**Aprobado**
PF-07	CU14 / RF23	Formular una pregunta abierta de ventas al asistente conversacional IA por chat web.	El LLM extrae la intención analítica hacia el catálogo de 23 métricas sin escribir código SQL directo.	Genera consulta parametrizada contra la BD del tenant y narra conclusiones gerenciales sin tecnicismos.	**Aprobado**
PF-08	CU15 / RF24	Exportar reporte de ventas dinámico a formato Excel nativo mediante botón de descarga.	Genera libro binario `.xlsx` con membrete de la tienda, formato `"Bs" #,##0.00` y fórmulas GAAP.	Descarga exitosa de archivo `.xlsx` legible en Microsoft Excel con auto-ajuste de columnas.	**Aprobado**

5.3. Pruebas de Integración

Las pruebas de integración evalúan el acoplamiento y la comunicación asíncrona entre el backend en NestJS, la base de datos PostgreSQL, el microservicio de Machine Learning en Python, la PWA en React 19 y las pasarelas externas:

Tabla 5.3: Registro de Pruebas de Integración

ID	Componentes Integrados	Escenario de Prueba	Resultado Esperado	Resultado Obtenido	Estado
PI-01	PWA (IndexedDB) $\leftrightarrow$ Backend NestJS $\leftrightarrow$ Tabla `idempotencia`	Reenviar una orden de venta offline ya sincronizada utilizando la misma cabecera `Idempotency-Key` (UUID).	El servidor detecta la llave en la tabla `idempotencia`, retorna HTTP 200 con `X-Cache-Idempotency: HIT` y no duplica stock.	Respuesta en 8 ms con cabecera HIT; inventario físico permanece inalterado.	**Aprobado**
PI-02	Frontend PWA $\leftrightarrow$ Backend NestJS $\leftrightarrow$ PostgreSQL (Concurrencia)	Dos clientes desconectados compiten por la última unidad disponible (`stock = 1`) al recuperar la red.	La primera orden es confirmada; la segunda recibe HTTP 409 `STOCK_AGOTADO_CONFLICTO` y activa compensación.	Servidor rechaza la 2.ª orden; PWA ejecuta rollback en outbox y levanta `ModalConflictoStock`.	**Aprobado**
PI-03	Backend NestJS $\leftrightarrow$ Microservicio `mirroria-ml` (FastAPI :8002)	Solicitar predicciones de demanda y no-show con el microservicio Python en ejecución.	Inferencia ejecutada por Random Forest y Logistic Regression con cabecera `X-ML-Fallback: false`.	Respuesta en 45 ms con métricas $R^2=0.9566$ y semáforos de asistencia calibrados.	**Aprobado**
PI-04	Backend NestJS $\leftrightarrow$ Fallback Heurístico (Microservicio Apagado)	Detener el contenedor `mirroria_ml` y solicitar predicciones desde el panel administrativo.	El interceptor detecta timeout/caída de red, ejecuta el cálculo heurístico y emite `X-ML-Fallback: true`.	Sistema entrega estimaciones matemáticas de cobertura sin interrumpir la interfaz web.	**Aprobado**
PI-05	Backend NestJS $\leftrightarrow$ Pasarela Banco Wayra (QR)	Simular callback de pago QR en `POST /pagos/qr/aviso` con firma criptográfica HMAC-SHA256.	Backend verifica firma con `crypto.timingSafeEqual`, asienta el cobro e impide reintentos con el mismo `event_id`.	Pago validado exitosamente; aviso duplicado es ignorado por restricción de unicidad.	**Aprobado**

5.4. Registro y Corrección de Errores

Durante las fases iterativas de desarrollo se registraron y subsanaron los siguientes defectos críticos:

Tabla 5.4: Registro de Defectos Corregidos

ID	Prueba Asociada	Descripción del Error	Gravedad	Causa Raíz Identificada	Corrección Técnica Aplicada	Estado
ERR-01	PI-01	Al reintentar sincronizar una orden offline con payload ligeramente modificado, el backend devolvía error interno 500.	**Alta**	No se comparaba el hash criptográfico del cuerpo contra la clave de idempotencia original.	Se incorporó el hashing SHA-256 canónico (`hash_cuerpo`), emitiendo HTTP 422 `UnprocessableEntity` ante discrepancias de payload.	**Corregido**
ERR-02	PI-02	Cuando el stock era 1 y ocurría un conflicto concurrente, la orden en la PWA permanecía bloqueada en estado `PENDIENTE`.	**Crítica**	El gestor outbox no capturaba el código de negocio `STOCK_AGOTADO_CONFLICTO`, tratándolo como fallo genérico de red.	Se añadió la transición explícita al estado `COMPENSADO` y el despacho del evento `mirroria:conflicto-stock` para abrir el modal guiado.	**Corregido**
ERR-03	PF-03	Al cambiar una tienda al modo oscuro, los textos principales perdían legibilidad sobre el fondo `#0A0A0A`.	**Media**	El contraste solo se validaba contra fondo blanco predeterminado.	Se actualizó `color.ts` para contrastar contra `FONDO_TIENDA[modo]` y se integró el ajustador perceptual OKLCH.	**Corregido**
ERR-04	PI-04	La caída del microservicio Python bloqueaba el hilo de NestJS durante 30 segundos antes de responder.	**Alta**	Las llamadas `fetch` nativas carecían de límite de tiempo de espera (*timeout*).	Se integró `AbortController` con un timeout estricto (hoy **1.500 ms** y una sola dirección) para conmutar de inmediato al fallback heurístico.	**Corregido**

5.5. Resumen Consolidado de Resultados

Tabla 5.5: Métricas Globales de Cobertura de Pruebas

Suite / Entorno de Pruebas	Total Ejecutadas	Aprobadas	Falli-das	Cobertura / Tasa de Éxito
Pruebas Unitarias Backend (Vitest)	557	557	0	**100%**
Pruebas de Integración e2e Backend	61	61	0	**100%** (Contra PostgreSQL local)
Pruebas Unitarias Frontend (Vitest)	345	345	0	**100%**
Pruebas App Móvil (Vitest + SQLite real)	17	17	0	**100%** (+1 de integración contra el backend, que corre aparte)
Pruebas Microservicio ML (Pytest)	6	6	0	**100%** (Pipelines M1-M4)
QA Automatizado de Aislamiento Tenant	84	84	0	**100%** (`qa_aislamiento.py`)
QA Automatizado de Reglas de Mostrador/POS	21	21	0	**100%** (`qa_reglas.py`)
QA Automatizado de Marca y Contraste WCAG	37	37	0	**100%** (`qa_marca.py`)
TOTAL CONSOLIDADO	**969**	**969**	**0**	**100% de Aprobación**

5.6. Casos de Prueba Detallados

A continuación se formaliza la especificación exhaustiva de dos casos de prueba neurálgicos para la defensa de grado:

Tabla 5.6.1: Caso de Prueba CP-01 — Aislamiento Multi-Tenant Estricto

Elemento	Especificación del Caso de Prueba
Identificador:	**CP-01**
Caso de Uso Asociado:	CU01 / CU02 — Aislamiento y Gobernanza Multi-Tenant
Objetivo:	Demostrar que un usuario autenticado en una boutique (`tiendaria`) no puede leer ni mutar información de otra tienda (`casa-andina`).
Precondiciones:	Existen dos tiendas activas con datos sembrados: `tiendaria` (Tenant A) y `casa-andina` (Tenant B).
Entradas:	Petición HTTP `GET /api/v1/ventas` y `PATCH /api/v1/catalogo/productos/:id` enviando el token JWT de `tiendaria` pero con la cabecera `X-Tenant: casa-andina`, o solicitando un UUID perteneciente a Tenant B.
Pasos de Ejecución:	1. Iniciar sesión con `admin@mirroria.com` obteniendo token JWT de `tiendaria`. 2. Despachar petición enviando cabecera `X-Tenant: casa-andina`. 3. Despachar petición `GET /catalogo/productos/<UUID_CASA_ANDINA>` sin cabecera cruzada.
Resultado Esperado:	En el paso 2: El interceptor detecta discordancia entre el JWT y la cabecera, respondiendo HTTP 403 `"Tu sesión pertenece a otra tienda"`. En el paso 3: La consulta ejecuta `where: soloTenant({...})`, afectando 0 filas y emitiendo HTTP 404 `"No encontrado"`.
Resultado Obtenido:	HTTP 403 en petición con cabecera ajena; HTTP 404 en consulta por ID foráneo. Cero fuga de información.
Estado Final:	**Aprobado con Distinción**

Tabla 5.6.2: Caso de Prueba CP-02 — Concurrencia Desconectada y Conflicto `stock = 1`

Elemento	Especificación del Caso de Prueba
Identificador:	**CP-02**
Caso de Uso Asociado:	CU10 / CU11 — Resiliencia Offline y Resolución de Conflictos
Objetivo:	Demostrar que ante dos compras offline simultáneas por la última prenda disponible, el sistema preserva la integridad del stock y activa compensación.
Precondiciones:	La variante SKU `VEST-ROJO-S` tiene exactamente `cantidad_disponible = 1` en la Sucursal Central.
Entradas:	Dos transacciones con `Idempotency-Key` diferenciados generadas sin red desde dos terminales (PWA y Móvil).
Pasos de Ejecución:	1. Desconectar ambas terminales de la red. 2. Registrar una venta de la prenda en la Terminal 1 y otra venta en la Terminal 2. 3. Ambas terminales descuentan provisionalmente su almacén local y encolan en outbox. 4. Restablecer la conexión en ambas terminales simultáneamente. 5. Terminal 1 drena su cola e impacta en el servidor. 6. Terminal 2 drena su cola milisegundos después e impacta en el servidor.
Resultado Esperado:	Terminal 1: El servidor confirma la venta, descuenta el stock a 0 y responde HTTP 201 (`C-XXXXXXXX`). Terminal 2: La sentencia `UPDATE` afecta 0 filas; el servidor responde HTTP 409 `STOCK_AGOTADO_CONFLICTO`. La Terminal 2 ejecuta rollback local y despliega `ModalConflictoStock` sugiriendo otras tiendas o tallas.
Resultado Obtenido:	Stock en PostgreSQL queda exactamente en 0 (cero inventario negativo); Terminal 1 recibe comprobante fiscal; Terminal 2 revierte su estado local a `COMPENSADO` y ofrece alternativas asistidas a la clienta.
Estado Final:	**Aprobado con Distinción**

CAPÍTULO VI. CONCLUSIONES Y RECOMENDACIONES

6.1. Conclusiones

Al culminar el ciclo de desarrollo y aseguramiento de calidad de la plataforma **MirrorIA**, se formulan las siguientes conclusiones de ingeniería:

1. **Evolución Exitosa hacia el Modelo SaaS Multi-tenant:**
   Se cumplió plenamente el objetivo central de evolucionar el prototipo original de tienda monousuario hacia una **plataforma SaaS multi-tenant de comercio electrónico de moda femenina**, capaz de aprovisionar boutiques independientes de manera atómica en una base de datos relacional compartida en PostgreSQL 16, garantizando el aislamiento lógico absoluto mediante el discriminador `tenant_id` y el filtrado obligatorio por request (`X-Tenant`).
2. **Eficacia y Trazabilidad de la Metodología ICONIX:**
   La adopción de la metodología **ICONIX** (*Doug Rosenberg & Matt Stephens*) permitió mantener una trazabilidad ininterrumpida y verificable entre los requisitos de negocio, los 17 casos de uso esenciales consolidados, los diagramas de robustez (BCE), los diagramas de secuencia UML 2.5 y las clases de diseño. El puente conceptual del análisis de robustez facilitó la detección temprana de inconsistencias antes de la fase de codificación, reduciendo drásticamente la deuda técnica.
3. **Resiliencia Operativa mediante Arquitectura Offline-First:**
   La implementación de capacidades desconectadas combinando **React 19 PWA** con **IndexedDB** en navegadores web y **React Native / Expo** con **SQLite** en dispositivos móviles demostró que es viable mantener la facturación en mostrador físico y la navegación de vitrinas ante caídas imprevistas de telecomunicaciones. La persistencia en almacenes locales sobrevive intacta al cierre de aplicación o reinicios de terminal.
4. **Garantía Transaccional e Idempotencia en Sincronización:**
   El protocolo de sincronización asíncrona respaldado por identificadores únicos **UUIDv4** y la cabecera canónica `Idempotency-Key` (con funciones de dispersión SHA-256) garantizó la estricta **idempotencia** de las transacciones, impidiendo dobles deducciones de inventario o duplicidad de cobros ante reintentos automáticos de red (`X-Cache-Idempotency: HIT`).
5. **Resolución Determinista del Conflicto de Concurrencia `stock = 1`:**
   Frente al problema clásico de colisión desconectada por la última unidad disponible en percha, el sistema demostró que la autoridad transaccional única del servidor mediante sentencias SQL `UPDATE` condicionadas (`WHERE cantidad_disponible >= 1`) combinada con respuestas HTTP 409 tipadas (`STOCK_AGOTADO_CONFLICTO`) permite a los clientes ejecutar transacciones compensatorias (rollback local) y guiar a la usuaria hacia alternativas viables (otras sucursales o tallas) sin generar frustración ni estados corruptos.
6. **Democratización de Experiencias Inmersivas y Accesibilidad Universal:**
   La integración del Probador Virtual por IA generativa (Decart `lucy-vton-3.5`) a través de credenciales efímeras seguras de 5 minutos, sumada a la validación matemática algorítmica de contraste accesible **WCAG**

2.1 ($\ge 4.5:1$) en espacio cromático OKLCH para cada boutique, posiciona a la plataforma a la vanguardia de la experiencia de usuario y cumplimiento ético de accesibilidad web.

7. **Analítica de Alto Impacto y Machine Learning Desacoplado:**
   La bifurcación de tableros de Business Intelligence (Consola Ejecutiva para directivos y Consola Operativa POS para cajeros), el confinamiento seguro de las consultas de DeepSeek LLM a un catálogo formal de 23 métricas parametrizadas (eliminando el riesgo de inyección o alucinación de SQL), la exportación nativa a hojas contables `.xlsx` (`exceljs`) y el despliegue del microservicio en Python FastAPI (:8002) con 4 modelos estadísticos reproducibles ($N=10.000$, `seed=42`) y **fallback heurístico resiliente** (`X-ML-Fallback: true`), dotan a las boutiques de herramientas predictivas de nivel corporativo para la prevención de quiebres de stock y mitigación de inasistencias en probador.

6.2. Recomendaciones

Con base en la experiencia obtenida durante la ejecución del proyecto y las oportunidades de optimización detectadas, se formulan las siguientes recomendaciones para futuras iteraciones del sistema:

1. **Validación Automática en Downgrade de Planes:**
   Implementar un asistente interactivo que valide que una boutique cumpla con las restricciones de recursos antes de permitir el descenso de categoría de plan (por ejemplo, exigir la despublicación voluntaria de prendas si se pasa de 300 a 50 productos activos al cambiar de Crecimiento a Básico).
2. **Facturación Electrónica en Línea y Hardware Fiscal:**
   Incorporar la integración nativa con los servicios web de Facturación en Línea del Servicio de Impuestos Nacionales (SIN) de Bolivia, permitiendo la firma digital de facturas XML y el soporte para impresión térmica directa en protocolos ESC/POS para impresoras de mostrador.
3. **Evolución hacia Arquitectura Dirigida por Eventos (EDA):**
   Para escenarios de alta concurrencia masiva (ej. campañas de Black Friday con miles de transacciones por segundo), se recomienda desacoplar la sincronización de la cola outbox mediante un bus de mensajería distribuido como **RabbitMQ** o **Apache Kafka**, permitiendo encolar las peticiones y procesarlas en micro-lotes en background.
4. **Gestión Avanzada de Conflictos Multivariable:**
   Expandir las capacidades de resolución de conflictos de stock en el cliente para permitir que el operador de caja modifique directamente la talla o aplique descuentos de cortesía sobre una prenda alternativa antes de reenviar la transacción rechazada.
5. **Paridad Total de Punto de Venta en la Aplicación Móvil:**
   Evolucionar el módulo móvil de Expo para incorporar el lector óptico de códigos de barra y QR mediante la cámara del celular, transformando cualquier teléfono móvil en una terminal POS móvil completa de inventario y caja para vendedores de piso.
6. **Profundización de Modelos de Visión Artificial:**
   Entrenar modelos locales de detección de puntos corporales (*pose estimation*) en el dispositivo móvil mediante TensorFlow Lite o CoreML, reduciendo el consumo de ancho de banda y la dependencia del servicio generativo en la nube para vistas previas rápidas en probador.

7. BIBLIOGRAFÍA Y 8. ANEXOS

7. BIBLIOGRAFÍA

- **Bass, L., Clements, P., & Kazman, R. (2021).** *Software Architecture in Practice* (4.ª ed.). Addison-Wesley Professional.
- **Bishop, C. M. (2006).** *Pattern Recognition and Machine Learning*. Springer.
- **Booch, G., Rumbaugh, J., & Jacobson, I. (2006).** *El Lenguaje Unificado de Modelado: Guía del Usuario* (2.ª ed.). Addison-Wesley.
- **Breiman, L. (2001).** Random Forests. *Machine Learning*, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324
- **Chong, F., & Carraro, G. (2006).** *Architecture Strategies for Catching the Long Tail: Software as a Service (SaaS)*. Microsoft Corporation.
- **Cox, D. R. (1958).** The Regression Analysis of Binary Sequences. *Journal of the Royal Statistical Society: Series B (Methodological)*, 20(2), 215–242.
- **Facebook Open Source. (2024).** *React 19 Documentation: The library for web and native user interfaces*. Meta Platforms, Inc. https://react.dev/
- **Hastie, T., Tibshirani, R., & Friedman, J. (2009).** *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2.ª ed.). Springer.
- **Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008).** Isolation Forest. En *Eighth IEEE International Conference on Data Mining (ICDM)* (pp. 413–422). IEEE. https://doi.org/10.1109/ICDM.2008.17
- **MacQueen, J. (1967).** Some Methods for Classification and Analysis of Multivariate Observations. En *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* (Vol. 1, pp. 281–297). University of California Press.
- **Martin, R. C. (2018).** *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- **NestJS Core Team. (2024).** *NestJS Documentation: A progressive Node.js framework for building efficient, reliable and scalable server-side applications*. https://docs.nestjs.com/
- **Object Management Group (OMG). (2017).** *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. Object Management Group. https://www.omg.org/spec/UML/2.5.1/
- **Python Software Foundation. (2024).** *Python 3.12 Documentation*. https://docs.python.org/3/
- **Ramírez, S. (Tiangolo). (2024).** *FastAPI: High performance, easy to learn, fast to code, ready for production*. https://fastapi.tiangolo.com/
- **Rosenberg, D., & Stephens, M. (2007).** *Use Case Driven Object Modeling with UML: Theory and Practice*. Apress. https://doi.org/10.1007/978-1-4302-0243-1
- **Stripe Inc. (2024).** *Stripe API Reference: Accept payments online and in-person*. https://stripe.com/docs/api

- **The PostgreSQL Global Development Group. (2024).** *PostgreSQL 16.5 Documentation*. https://www.postgresql.org/docs/16/
- **World Wide Web Consortium (W3C). (2018).** *Web Content Accessibility Guidelines (WCAG) 2.1: W3C Recommendation*. https://www.w3.org/TR/WCAG21/
- **World Wide Web Consortium (W3C). (2015).** *Indexed Database API (IndexedDB): W3C Recommendation*. https://www.w3.org/TR/IndexedDB/

---

8. ANEXOS

Anexo A: Catálogo Histórico de Casos de Uso del 1.er Ciclo (Línea Base PUDS)

Para garantizar la consistencia longitudinal del proyecto, se incorpora la matriz de casos de uso funcionales completados durante la primera entrega (CU01 a CU24), hoy plenamente integrados en el espacio de trabajo de la boutique oficial **Tiendaria** (`/t/tiendaria`):

Código	Nombre del Caso de Uso	Actores Primarios	Propósito Funcional	Prioridad Inicial
CU01	Registrar Usuario	`CUSTOMER`	Autocreación de cuentas de clientas en la boutique con email único.	Alta
CU02	Iniciar Sesión	Todos los Actores	Verificación de credenciales BCrypt y emisión de Access Token JWT.	Alta
CU03	Gestionar Usuarios y Roles	`ADMIN`	Asignación de roles institucionales (`CAJERO` , `ENCARGADO`) y sucursales.	Alta
CU04	Gestionar Proveedores	`ADMIN`	Alta de empresas confeccionistas y talleres de suministro textil.	Media
CU05	Gestionar Categorías	`ADMIN`	Taxonomía jerárquica de prendas con slug amigable para la web.	Alta
CU06	Gestionar Temporadas y Colecciones	`ADMIN`	Parametrización de colecciones vinculadas obligatoriamente a talleres.	Media
CU07	Gestionar Tallas y Colores	`ADMIN`	Catálogo normalizado de tallas ordenadas y colores con código HEX.	Alta
CU08	Gestionar Productos y Variantes	`ADMIN`	Publicación de prendas maestras con imágenes y combinaciones SKU.	Alta
CU09	Explorar Catálogo de Prendas	`CUSTOMER` , Visitante	Navegación de vitrinas, filtrado por categorías y fichas técnicas.	Alta
CU10	Consultar Stock por Sucursal	`CUSTOMER` , `CAJERO`	Verificación de existencias físicas disponibles en cada tienda física.	Alta
CU11	Gestionar Carrito de Compras	`CUSTOMER`	Adición de variantes a la bolsa `jsonb` con tope de 3 unidades.	Alta
CU12	Procesar Checkout Digital	`CUSTOMER`	Conversión del carrito a venta `PENDIENTE` con descuento atómico.	Alta
CU13	Registrar Venta Presencial en Caja	`CAJERO`	Venta física inmediata en mostrador emitiendo comprobante fiscal.	Alta
CU14	Consultar Historial de Compras	`CUSTOMER` , `ADMIN`	Vista detallada de pedidos y comprobantes emitidos en "Mis pedidos".	Media
CU15	Gestionar Ciudades y Sucursales	`ADMIN`	Alta de puntos de venta físicos en las capitales de Bolivia.	Alta
CU16	Gestionar Órdenes de Compra	`ADMIN`	Emisión de pedidos de confección incrementando el stock en tránsito.	Media
CU17	Recepcionar Pedido de Proveedor	`ENCARGADO_SUCURSAL`	Llegada de mercancía a tienda aumentando la `cantidad_disponible` .	Alta
CU18				Media

Código	Nombre del Caso de Uso	Actores Primarios	Propósito Funcional	Prioridad Inicial
	Ajustar Stock Manualmente	`ENCARGADO`, `ADMIN`	Corrección de existencias por mermas con justificación explícita.	
CU19	Auditar Kardex de Movimientos	`ADMIN`, `ENCARGADO`	Traza inmutable de movimientos (`VENTA`, `RESERVA`, `RECEPCION`, `AJUSTE`).	Alta
CU20	Solicitar Reserva en Tienda	`CUSTOMER`	Bloqueo temporal de prendas en `cantidad_reservada` para probador.	Alta
CU21	Gestionar Estado de Reserva	`ENCARGADO_SUCURSAL`	Avance del ciclo de vida de la cita (`CONFIRMADA`, `EN_TIENDA`).	Alta
CU22	Cobrar Reserva en Punto POS	`CAJERO`	Cierre de reserva en mostrador mediante venta física y liberación de stock.	Alta
CU23	Cancelar Reserva de Prendas	`CUSTOMER`, `ENCARGADO`	Anulación de cita con restitución inmediata de unidades a stock libre.	Media
CU24	Generar Reportes Dinámicos por IA	`ADMIN`, `ENCARGADO`	Formulación de consultas en lenguaje natural (texto o voz) con LLM.	Alta

---

Anexo B: Enlaces Oficiales del Repositorio y Despliegue en la Nube

Para facilitar la verificación de los resultados por parte de la docente y el tribunal evaluador, se suministran los accesos oficiales a los artefactos de software:

- **Repositorio de Código Fuente (Monorepo GitHub):**
  https://github.com/mopy1/MirrorIA
  Estructura monorepo con `mirroria-backend/`, `mirroria-frontend/`, `mirroria-ml/`, `scripts/` y `db/`.
- **Despliegue en Producción (Máquina Virtual en Azure Cloud; se redespliega con `scripts/deploy-azure.sh` antes de la defensa):**
  https://mirroria.duckdns.org
  Despliegue asegurado con certificados TLS/SSL válidos de Let's Encrypt sobre Nginx reverse proxy, PostgreSQL 16 y Docker.
- **Documentación Interactiva de la API (Swagger UI):**
  `http://localhost:3000/api/docs` *(Habilitado en entornos de desarrollo y evaluación local).*